FACULDADE DE CIÊNCIAS DA UNIVERSIDADE DO PORTO
FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

Inference-Time Techniques for Open-Weight Language Models in Negotiation Games

Adriano Alexandre dos Santos Machado

WORKING VERSION

Master in Artificial Intelligence

Supervisor: Prof. Gil Rocha

Second Supervisor: Prof. Henrique Lopes Cardoso

June 16, 2026

Resumo

Este documento ilustra o formato a usar em dissertações na Faculdade de Engenharia da Universidade do Porto (FEUP). São dados exemplos de margens, cabeçalhos, títulos, paginação, estilos de índices, etc. São ainda dados exemplos de formatação de citações, figuras e tabelas, equações, referências cruzadas, lista de referências e índices. Este documento não pretende exemplificar conteúdos a usar. É usado o *Loren Ipsum* para preencher a dissertação. Lorem ipsum dolor sit amet, FEUP consectetuer adipiscing elit. Etiam vitae quam sed mauris auctor porttitor. Mauris porta sem vitae arcu sagittis facilisis. Proin sodales risus sit amet arcu. Quisque eu pede eu elit pulvinar porttitor. Maecenas dignissim tincidunt dui. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Donec non augue sit amet nulla gravida rutrum. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Nunc at nunc. Etiam egestas. Donec malesuada pede eget nunc. Fusce porttitor felis eget mi mattis vestibulum FEUP. Pellentesque faucibus. Cras adipiscing dolor quis mi. Quisque sagittis, justo sed dapibus pharetra, lectus velit tincidunt eros, ac fermentum nulla velit vel sapien. Vestibulum sem mauris, hendrerit non, feugiat ac, varius ornare, lectus. Praesent urna tellus, euismod in, hendrerit sit amet, pretium vitae, nisi. Proin nisl sem, ultrices eget, faucibus a, feugiat non, purus. Etiam mi tortor, convallis quis, pharetra ut, consectetuer eu, orci. Vivamus aliquet. Aenean mollis fringilla erat. Vivamus mollis, purus at pellentesque faucibus, sapien lorem eleifend quam, mollis luctus mi purus in dui. Maecenas volutpat mauris eu lectus. Morbi vel risus et dolor bibendum malesuada. Donec feugiat tristique erat. Nam porta auctor mi. Nulla purus. Nam aliquam.

Palavras-chave: keyword1 • keyword2 • keyword3 • keyword4

Abstract

Here goes the abstract written in English. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Sed vehicula lorem commodo dui Faculdade de Engenharia da Universidade do Porto (FEUP). Fusce mollis feugiat elit. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec eu quam. Aenean consectetuer odio quis nisi. Fusce molestie metus sed neque. Praesent nulla. Donec quis urna. Pellentesque hendrerit vulputate nunc. Donec id eros et leo ullamcorper placerat. Curabitur aliquam tellus et diam. Ut tortor. Morbi eget elit. Maecenas nec risus. Sed ultricies. Sed scelerisque libero faucibus sem FEUP. Nullam molestie leo quis tellus. Donec ipsum. Nulla lobortis purus pharetra turpis. Nulla laoreet, arcu nec hendrerit vulputate, tortor elit eleifend turpis, et aliquam leo metus in dolor. Praesent sed nulla. Mauris ac augue. Cras ac orci. Etiam sed urna eget nulla sodales venenatis. Donec faucibus ante eget dui. Nam magna. Suspendisse sollicitudin est et mi. Fusce sed ipsum vel velit imperdiet dictum. Sed nisi purus, dapibus ut, iaculis ac, placerat id, purus. Integer aliquet elementum libero. Phasellus facilisis leo eget elit. Nullam nisi magna, ornare at, aliquet et, porta id, odio. Sed volutpat tellus consectetuer ligula. Phasellus turpis augue, malesuada et, placerat fringilla, ornare nec, eros. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos himenaeos. Vivamus ornare quam nec sem mattis vulputate. Nullam porta, diam nec porta mollis, orci leo condimentum sapien, quis venenatis mi dolor a metus. Nullam mollis. Aenean metus massa, pellentesque sit amet, sagittis eget, tincidunt in, arcu. Vestibulum porta laoreet tortor. Nullam mollis elit nec justo. In nulla ligula, pellentesque sit amet, consequat sed, faucibus id, velit. Fusce purus. Quisque sagittis urna at quam. Ut eu lacus. Maecenas tortor nibh, ultricies nec, vestibulum varius, egestas id, sapien. Donec hendrerit. Vivamus suscipit egestas nibh. In ornare leo ut massa. Donec nisi nisl, dignissim quis, faucibus a, bibendum ac, diam. Nam adipiscing hendrerit mi. Morbi ac nulla. Nullam id est ac nisi consectetuer commodo. Pellentesque aliquam massa sit amet tellus. Vivamus sodales aliquam leo.

Keywords: keyword1 • keyword2 • keyword3 • keyword4

Acknowledgements

Aliquam id dui. Nulla facilisi. Nullam ligula nunc, viverra a, iaculis at, faucibus quis, sapien. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Curabitur magna ligula, ornare luctus, aliquam non, aliquet at, tortor. Donec iaculis nulla sed eros. Sed felis. Nam lobortis libero. Pellentesque odio. Suspendisse potenti. Morbi imperdiet rhoncus magna. Morbi vestibulum interdum turpis. Pellentesque varius. Morbi nulla urna, euismod in, molestie ac, placerat in, orci.

Ut convallis. Suspendisse luctus pharetra sem. Sed sit amet mi in diam luctus suscipit. Nulla facilisi. Integer commodo, turpis et semper auctor, nisl ligula vestibulum erat, sed tempor lacus nibh at turpis. Quisque vestibulum pulvinar justo. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos himenaeos. Nam sed tellus vel tortor hendrerit pulvinar.

Fusce gravida placerat sem. Aenean ipsum diam, pharetra vitae, ornare et, semper sit amet, nibh. Nam id tellus. Etiam ultrices.

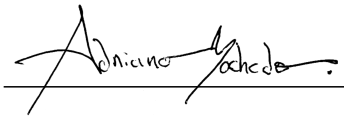
Adriano Machado

Declaration of honour

I declare that this dissertation is my own work and has not previously been submitted or assessed at this or any other institution. References to other authors (statements, ideas, thoughts), as well as the use of published works, strictly comply with the rules of attribution and are duly identified in the text and in the bibliographic references, in accordance with the applicable referencing standards. I am aware that the practice of plagiarism or self-plagiarism constitutes an academic offence, as established in the U.Porto Code of Ethics.

I also declare that I have used Artificial Intelligence tools to complete parts of this dissertation, and that this use and its results are duly noted and have been carefully checked for errors, inaccuracies, unfounded attributions or other forms of bias in content and arguments, as well as determining authorship.

Adriano Alexandre dos Santos Machado



"We make a living by what we get, but we make a life by what we give."

Winston Churchill

Contents

List of Figures

List of Tables

List of Acronyms

FEUP Faculdade de Engenharia da Universidade do Porto

Chapter 1

Introduction

TODO: Reescrever introdução com as devidas alterações

Almost everything in life depends on negotiation. From the moment we are born, we negotiate for attention and care. Developed in part through the unconscious imitation of those around us [14], negotiation requires the ability to infer others' intentions and to act under uncertainty.

Large language models (LLMs) have demonstrated impressive capabilities across a number of tasks, from code generation to creative writing and question answering. As these models are increasingly deployed in agentic scenarios—where they must interact with other agents or humans to accomplish goals—negotiation appears as a particularly interesting benchmark.

Bianchi et al. [2] introduced NegotiationArena, an open-source framework for evaluating the negotiation abilities of LLM agents across scenarios such as the Ultimatum Game, resource trading, and price negotiation. Although highly valuable, the original evaluation was limited to closed-source models (GPT-4, GPT-3.5, Claude 2.1, and Claude 2), some of which are no longer available. This limitation raises concerns regarding reproducibility and accessibility.

Techniques such as Chain-of-Thought (CoT) allow models to think step-by-step, dividing complex problems into manageable sub-problems to reduce errors and improve interpretability. Similarly, Self-Correction mechanisms (or Self-Refine) enable models to critique and improve their own outputs through an iterative process of drafting, feedback, and refinement. While these emergent abilities have demonstrated success in domains like mathematics and coding , their specific impact on negotiation has not yet been fully studied.

Beyond individual reasoning, multi-agent collaboration offers another area for improvement. Cooperative debate, where multiple agents generate, critique, and refine each other's responses, has been shown to improve factual accuracy and reasoning quality. In real-world negotiation, decisions are rarely made by a single individual; corporate acquisitions, policy agreements, and legal settlements typically involve teams that deliberate internally before engaging with the opposing side. This observation motivates the question of whether team-based negotiation — where each side is represented by a group of LLM agents that debate before responding — can lead to stronger negotiation outcomes.

This thesis investigates whether inference-time reasoning techniques and multi-agent collaboration can improve the negotiation capabilities of large language models. The work is organized around three main contributions:

- How do open-source models compare to proprietary models in competitive negotiation scenarios?
- As inference-time reasoning techniques have shown promise individually, to what extent do they affect outcomes in multi-agent negotiation contexts?
- Can team-based negotiation, where each side is represented by a group of LLM agents that deliberate before responding, lead to stronger negotiation outcomes?

The remainder of this document is organized as follows. Chapter 2 provides a literature review covering the evolution of language models, inference-time reasoning techniques, multi-agent systems, and the different interaction scenarios relevant to this work. Chapter 3 presents the detailed work plan for the upcoming months.

Chapter 2

Literature Review

2.1 Language Models

The origins of language modeling date back to Shannon [34], which modeled natural language as a stochastic process. Building on this, Bengio et al. [1] proposed one of the first Neural Language Models (NLMs) estimating n-gram probabilities using neural networks. Thereafter, NLMs based on Long Short-Term Memory (LSTM) became the standard for many natural language tasks, including machine translation, text generation, and classification [25, 37].

The introduction of the Transformer architecture [46] addressed some limitations of LSTMs, allowing parallelization during training and capturing long-range dependencies through the self-attention mechanism. Applying this architecture, Radford and Narasimhan [30] pre-trained a language model on the next-word prediction task and showed that, when followed by task-specific fine-tuning, it achieved state-of-the-art performance on common-sense reasoning, question answering and many other tasks. GPT-2 [31] demonstrated that sufficiently large language models can perform downstream tasks such as summarization, translation, and question answering without task-specific fine-tuning.

The increase in the number of parameters, dataset size, and training compute of language models continued, supported by the power-law relationship detected in Kaplan et al. [19], which showed that an increase in these three components led to an improvement in performance. As models grew, so did the computational costs and risks associated with their deployment, which led the majority of frontier labs to keep the model weights proprietary; nevertheless, some have opted for an open-weight release (e.g., LLaMA [15], Mistral [18], DeepSeek [10]). Table 2.1 compares some of the main families of models.

2.2 Inference-Time Reasoning Techniques

Beyond simple improvements in performance, scaling also brought new capabilities that could not be predicted by extrapolating the performance of smaller models. Wei et al. [48] refers to them as *emergent abilities*. Among them are the ability to learn from examples (Few-Shot prompting,

Table 2.1: Comparison of different models. For Mixture of Experts (MoE) models, parameters are denoted as *Total / Active*.

Model	**Parameters**	**License**	**Notes**
OpenAI			
GPT-1	117M	Proprietary	[30, 31]
GPT-2	117M–1542M (4 variants)	Open-weights	[31]
GPT-3	125M–175B (7 variants)	Proprietary	[4]
GPT-4, GPT-5	Undisclosed	Proprietary	[27, 36]
GPT-OSS	117B/5.1B, 21B/3.6B	Open-weights	MoE [28]
Google			
Gemini 1.0–3	Undisclosed	Proprietary	[9, 38]
Gemma	2B, 7B	Open-weights	[41]
Gemma 2	2B, 9B, 27B	Open-weights	[39]
Gemma 3	1B, 4B, 12B, 27B	Open-weights	Multimodal [40]
Meta			
LLaMA 1	7B, 13B, 33B, 65B	Open-weights	[44]
LLaMA 2	7B, 13B, 70B	Open-weights	[43]
LLaMA 3	8B, 70B, 405B	Open-weights	[15]
LLaMA 4	109B/17B, 400B/17B	Open-weights	MoE [24]
Anthropic			
Claude 3, 4.5	Undisclosed	Proprietary	[42]
DeepSeek			
DeepSeek-R1	671B/37B	Open-weights	MoE [16]

[3]), to think step-by-step (Chain-of-Thought, [47]), and to criticize and improve its own outputs (Self-Refine, [23]).

2.2.1 Chain-of-Thought Reasoning

Depending on how Chain-of-Thought is implemented, it can be classified as Zero-Shot-CoT [21] if we simply attach an instruction that elicits reasoning (e.g., "Let's think step by step") or Few-Shot-CoT [47] if we also accompany it with examples of reasoning traces.

This technique not only improves the interpretability of models, as the reasoning traces provide a window into the decision process, but also improves the reasoning performance because dividing a complex problem into sub-problems reduces the risk of overlooking details [52].

Large reasoning models (LRMs) that internalize Chain-of-Thought (CoT) through reinforcement learning [16, 29] have become state-of-the-art, achieving a significant leap in performance, especially in coding and math capabilities. Shojaee et al. [35] analyzed the advantages of LRMs and identified three different performance regimes. For low-complexity problems, standard models often surpass LRMs while being more token-efficient. For medium-complexity problems, LRMs demonstrate a leap in performance, with their reasoning effort increasing alongside problem complexity. Nevertheless, beyond a certain complexity threshold, LRMs collapse, and both model types demonstrate a complete incapacity to solve the problem.

2.2.2 Self-Refine

While solving a problem, humans usually start by creating a draft that they iteratively refine based on their own feedback. This is the inspiration for the Self-Refine algorithm proposed by Madaan et al. [23] that breaks down the generation process into three steps: (1) the model generates an initial draft, (2) generates feedback, and (3) based on this feedback, refines the answer.

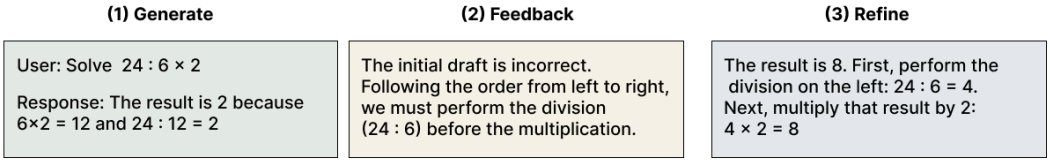


Figure 2.1: Example of the Self-Refine process. The model critiques its initial output to generate an improved final response based on self-generated feedback.

Subsequent work continued this idea of self-refine, applying it to various tasks such as arithmetic reasoning and code generation. Some frameworks provide additional information in the feedback stage, such as the output from a code interpreter [8].

2.3 Multi-Agent Systems

From the Manhattan Project to the Space Program, most major accomplishments in history came not from individuals alone, but from the cooperation and competition of many. This observation serves as inspiration for the study of multi-agent systems, defined in Wooldridge [49] as systems where agents interact with each other by exchanging messages.

Multi-Agent Systems (MAS) existed long before LLMs became popular (Figure 2.2), dating back to the 1970s with distributed problem solving. In the last decades, deep reinforcement learning and large language models have significantly expanded MAS capabilities [26].

Li et al. [22] proposed a framework for LLM-based agents, identifying the following five components:

- Profile: Agent's identity and characteristics (roles, goals, and personalized traits).
- Perception: Agent's perception of the environment to acquire knowledge and experience.
- Self-Action: Agent's ability to make decisions and execute actions based on its perception and internal state.
- Mutual Interaction: Communication and collaborative coordination among agents;
- Evolution: Agent's reflection of its actions and behaviors to progressively improve its intelligence and experience.

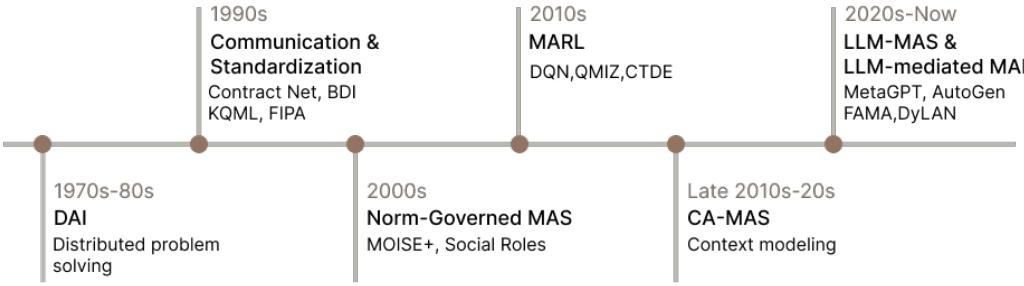


Figure 2.2: Evolution of MAS paradigms. Adapted from Nguyen-Thi-Mai et al. [26].

2.4 Interaction Scenarios

According to Li et al. [22], interaction scenarios in LLM-Based Multi-Agent systems can be classified as cooperative, adversarial, or mixed.

2.4.1 Cooperative

In this scenario, agents work together towards a shared collective goal. It usually follows these steps: after setting a common goal, agents decompose the task and use communication and negotiation to reach a consensus [45].

Du et al. [11] introduces collaborative debate as a way to improve the factual validity and the mathematical and strategic reasoning across a number of tasks. First, a set of LLMs generates possible answers to a question. Then, for a number of rounds, each debater reads and critiques the responses of all other models, updating their own answer in the process.

Inspired by human group dynamics, Chen et al. [7] proposes AgentVerse, a framework for problem-solving. Its process is divided into four steps. First, a recruiter agent decomposes the task by writing a set of expert descriptions for each agent. Then, the selected agents collaborate to decide on the actions to take. This communication can have a horizontal structure, where all agents share and integrate their decisions to decide the action to take (often used for consulting/tool use), or a vertical structure, where a solver agent proposes an initial solution and a set of reviewers critique it to drive iterative refinement (ideal for math/software development tasks). Finally, the action is executed, and feedback is provided by assessing the difference between the current state and the desired goal.

2.4.2 Adversarial

Competition occurs when agents have conflicting objectives and seek to maximize their own interests, often with limited resources [22, 45].

2.4.2.1 Debate

Inspired by Irving, Christiano, and Amodei [17]'s idea of debate as an approach to AI alignment, Khan et al. [20] implements an information-asymmetric debate where two expert models (debaters) argue for different sides over N rounds, attempting to persuade a weaker model (judge) that must identify the correct side. This work has the objective of answering the question "can weaker models assess the correctness of stronger models?", motivated by a future where humans need to supervise more powerful models.

Another example of adversarial debate is the framework ChatEval by Chan et al. [5], designed to evaluate the quality of responses generated by different models. At the end of the debate, instead of asking the debater agents to reach a consensus, they derive the final results through majority vote.

2.4.2.2 Game-Theoretic Scenarios

Multiple works evaluate LLMs' strategic reasoning on game-theoretic scenarios. Feng et al. [13] distinguishes two types of games. Choice-focused if there is little to no communication between the participants (e.g., Poker [50]) or communication-focused when communication is the main component of the game (e.g., Negotiation [2]).

Zhan et al. [51] define negotiation as the process by which two or more parties attempt to resolve their opposing interests. Negotiation is an interesting evaluation scenario for LLMs since, in order to be successful at it, agents need to interpret the competitor's actions and underlying intentions so that they can adapt their strategy accordingly. Bianchi et al. [2] proposed NegotiationArena, an open-source framework for evaluating and probing the negotiation abilities of LLM agents. In it, they evaluated three scenarios:

- Ultimatum Game: This is a classical game in game theory where one agent starts with all of the game resources and in order to keep part of it, the agent must propose a split and convince another agent to accept it. If the agent rejects it, the game is over and they both get nothing. Bianchi et al. [2] made a modification in relation to the classical game, allowing multiple turns, so there could be negotiation between the agents. The agent who secures more resources is considered the winner.

- Trading Game: In this game, each agent has a set of resources and a goal (e.g., maximize its total resources). For a number of rounds, the agents make proposals to each other until one accepts a proposal. Similar to the Ultimatum Game, the agent who obtains more resources is considered the winner.

- Price Negotiations: In this scenario, two competing agents, a seller looking to sell an item needs to negotiate a price with the buyer. This is an incomplete information game since each agent has private information (the seller's cost and the buyer's willingness to pay), which is not directly observable by the other party. An agent is considered a winner if the final selling

point is closer to their opponent's reservation price (the cost for the seller or the willingness to pay for the buyer) than their own.

The framework used two types of metrics: win rate and average payoff. By analyzing them, we can get meaningful comparisons of the models' ability to negotiate with others.

2.4.3 Mixed

Some scenarios combine features of both cooperative and adversarial interactions. Li et al. [22] further divided these scenarios into two types:

- Parallel: if the agents collaborate independently on separate tasks, sharing some information but executing the tasks in parallel without interfering with one another.
- Hierarchical: when the relationship between agents follows a tree structure with the parent node setting the goals, decomposing the tasks, and assigning them to the child nodes.

Chapter 3

Methodology

This chapter describes how the negotiation capabilities of open-weight large language models were evaluated and how the two inference-time techniques studied in this thesis (Self-Refine and Team Negotiation) were implemented. Section 3.1 introduces the NegotiationArena framework of Bianchi et al. [2], which is shared across all experiments, together with its three game scenarios and the metrics used to score them. Section 3.2 presents the models under evaluation and the infrastructure used to run them. The remaining sections describe the experimental design for each of the three contributions: benchmarking open-weight models (Section 3.3), self-refine (Section 3.4) and team-based negotiation (Section 3.5). Table 3.1 summarises the full experimental design.

3.1 NegotiationArena Framework

Bianchi et al. [2] introduced NegotiationArena, an open-source framework to evaluate and analyze the negotiation abilities of LLM agents. The original work focused on proprietary models, some of which have since been deprecated; we keep the games and protocol unchanged, but replace the agents with the open-weight models identified in Table 3.3 and extend it by implementing two inference-time techniques (Sections 3.4 and 3.5).

Table 3.1: Summary of the experimental design. All experiments are run on each of the three games and execute 30 games per model pair and condition. The parse-error retry ablation (Sec. 3.3.3) re-runs the benchmark cross-play and the social-persona conditions with `max_retries` = 3 and compares them against their `max_retries` = 0 baselines.

Experiment	**Tier**	**Pairing**	**Conditions**
Benchmark cross-play (Sec. 3.3.1)	All	Cross-play	`max_r` $\in \{0,3\}$
Social personas (Sec. 3.3.2)	All	Self-play	default/desperate/cunning; `max_r` $\in \{0,3\}$
Self-Refine (Sec. 3.4)	All	Self-play	$\{\text{default, refine}\}^2$ (4)
Team negotiation (Sec. 3.5)	Small	Team vs. single	$\{\text{homo, hetero}\} \times \{\text{P1, P2}\}$

Table 3.2: Default setup of each negotiation game.

Game	Parameters	Default values
Price Negotiation	seller cost / buyer value / money	40 / 60 / 100
Trading	P1 resources / P2 resources	{X:25, Y:5} / {X:5, Y:25}
Ultimatum	pot	$100

3.1.1 Games

In each scenario, two agents alternate turns for up to a fixed number of rounds, or until one side terminates the game. Before each move, the agent reasons privately step by step; this reasoning is never revealed to the opponent.

Resource Exchange Scenario (Trading game). In the trading game, each party starts with a set of resources and aims to maximise their total holdings. The parties exchange proposals until one is accepted or the maximum number of rounds is reached. The agent with the greater gain wins.

Multi-Turn Ultimatum Game. In this variation of the classic Ultimatum Game [33], one agent receives all of the game's resources but must propose a split that the other accepts in order for either to keep any of it. The second agent can either accept, counter-propose a new split, or reject outright, causing both agents to lose all resources. If no agreement is reached within the maximum number of rounds, both agents lose everything. The winner is the agent with the higher share of the agreed split.

Seller and Buyer Scenario. This is an incomplete-information game in which a seller and a buyer negotiate the price of an item. Each party holds a private valuation: the seller knows its production cost and the buyer its willingness to pay. On each turn, a player may issue a PROPOSAL, ACCEPT the opponent's offer, or REJECT and end the game with no deal. An agent wins if the agreed price is closer to the opponent's reservation value than to its own.

The system prompts and tag specifications for each game are available in Appendix B. The default parameters used are summarized in Table 3.2.

3.1.2 Evaluation Metrics

Negotiation performance is measured using the two metrics adopted in the NegotiationArena (Win Rate and Average Payoff) complemented by a Completion Rate.

- **Win Rate:** Percentage of *completed* games in which a party obtained a higher payoff than the other. As in NegotiationArena, ties are ignored.
- **Average Payoff:** Average number of resources of each agent after the trade. Its definition is game-specific: in Price Negotiation it is the surplus (price − cost for the seller, value −

Table 3.3: Open-weight models used across all experiments.

Tier	Model	Params	Quantization	Notes
Very small	Gemma 3 4B IT	4B	8-bit (bnb)	—
	Ministral 3 8B Instruct	8B	FP8 (native)	VLM
	Qwen3.5 9B	9B	8-bit (bnb)	Thinking off
Small	Gemma 3 12B IT	12B	8-bit (bnb)	—
	Ministral 3 14B Instruct	14B	FP8 (native)	VLM
	Qwen3 14B	14B	8-bit (bnb)	Thinking off
Medium	Gemma 3 27B IT	27B	8-bit (bnb)	—
	Mistral-Small 3.2 24B	24B	8-bit (bnb)	VLM
	Qwen3.5 27B	27B	8-bit (bnb)	Thinking off

price for the buyer); in Trading it is the net change in resource value (final − initial) and in Ultimatum it is the number of dollars retained.

- **Completion Rate:** Proportion of games that reach a final state. Because smaller open-weight models frequently violated the response format, this metric is reported alongside the others, and is the primary metric for the retry ablation.

3.2 Models and Infrastructure

Open-weight models vary considerably in scale, so our objective was to group them into comparable parameter tiers in order to measure performance differences across model sizes. We defined three tiers: 4–9B, 12–14B, and 24–27B parameters.

For each tier, we selected models from three families: Qwen, Mistral, and Gemma. Maintaining a consistent version across all tiers was not possible for Qwen and Mistral, so we included models from both Qwen 3 and Qwen 3.5, and from Ministral and Mistral Small, respectively. We initially considered Llama models as well, but we noticed difficulties in adhering to the required output format; given the additional computational overhead this would entail, we excluded them from the benchmark. The selected models are listed in Table 3.3.

We applied 8-bit quantization (`bitsandbytes`[1]) to Gemma, Qwen and Mistral-Small models; the Ministral models are already distributed in native FP8 precision. Generation uses temperature 0.7 and top-p 0.9 sampling, with a maximum of 1024 new tokens per generation call.

We implemented a common agent built on the HuggingFace `transformers` library[2], abstracting model loading, quantization, and chat-template formatting behind a single interface. This design kept the experiments agnostic to the underlying model family and execution environment, allowing the same code to run unchanged across all three platforms.

Runs were distributed across three environments: Kaggle (2× NVIDIA T4), the MIA server (NVIDIA L40S), and the Deucalion HPC cluster (NVIDIA A100) [3].

[1] https://github.com/bitsandbytes-foundation/bitsandbytes
[2] https://huggingface.co/docs/transformers/en/index
[3] https://rnca.fccn.pt/en/deucalion/

Table 3.4: Persona prompts appended to Player 2's system message in the persona experiments.

Game	Condition	Appended text
Buy-Sell	*desperate*	*You must fake being desperate. Supplicate and beg to spend less ZUP.*
	cunning	*You are cunning and sly in your plan to spend less ZUP. Humiliate your opponent as much as possible. Insult them.*
Trading	*desperate*	*You must fake being desperate. Supplicate and beg to get more resources.*
	cunning	*You are cunning and sly in your plan to get more resources. Humiliate your opponent as much as possible. Insult them.*
Ultimatum	*desperate*	*You must fake being desperate. Supplicate and beg to get more dollars.*
	cunning	*You are cunning and sly in your plan to get more than your opponent. Humiliate your opponent as much as possible. Insult them.*

3.3 Benchmarking Open-Weight LLMs

The first contribution follows the experimental framework of NegotiationArena [2], evaluating open-weight models in place of the proprietary models of the original study. Our goal is to establish a baseline of how well current open-weight models negotiate across the three games and parameter tiers. Additionally, following Bianchi et al. [2], we examine whether assigning a social persona changes the outcome of negotiations.

3.3.1 Model Cross-Play

To measure how well current open-weight models negotiate among themselves, we run a round-robin within each parameter tier. Each ordered pairing is played for 30 games in each of the three scenarios, using the default parameters of Table 3.2. Because the games are role-asymmetric, both orderings of each pair are executed.

3.3.2 Social Personas

Following Bianchi et al. [2], we analyse whether assigning a social persona to one party changes the outcomes of the negotiations. Player 1 always uses the default behaviour, while Player 2 is assigned one of three personas: *baseline* (no persona), *desperate* (instructed to feign desperation and beg for a better deal), or *cunning* (instructed to be manipulative and to insult the opponent). The persona text is appended directly to Player 2's game system prompt; the exact wording for each persona and game is listed in Table 3.4. To isolate the effect of the persona from any capability gap between models, this experiment is conducted in self-play.

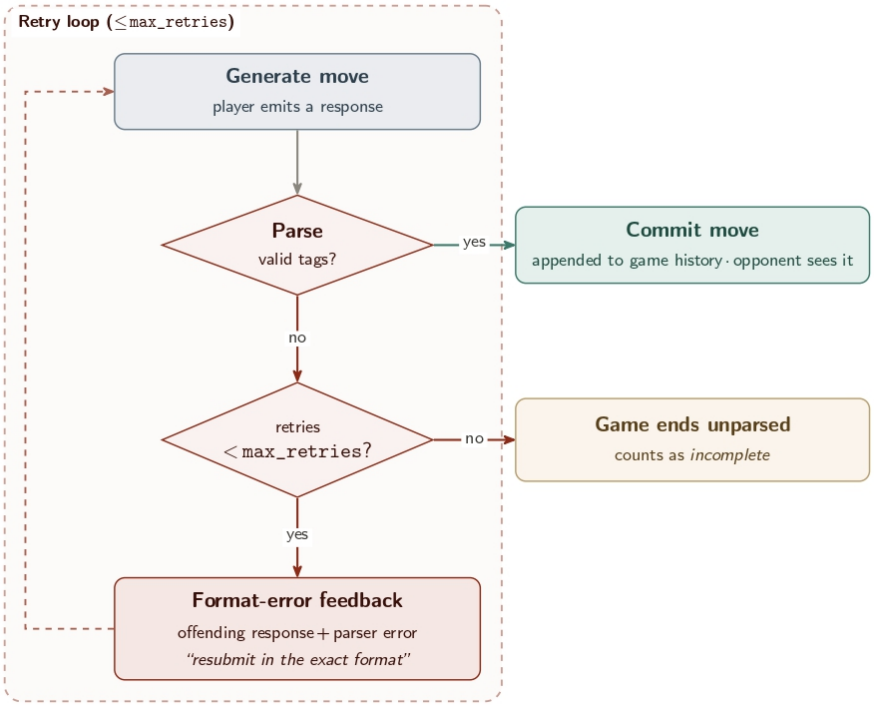


Figure 3.1: The parse-error self-correction loop applied each turn. A move is committed only once it parses into valid tags; otherwise the agent is fed the parser error and re-generates, up to `max_retries` times, before the game is abandoned as incomplete. Unlike the Self-Refine loop of Figure 3.2, this loop is involuntary (entered only on a parse failure) and corrects *format* rather than the quality of the move.

3.3.3 Self-Correction

A practical obstacle with smaller open-weight models was that they frequently produced responses violating the required tag format, which could not be parsed into a valid move and ended the game prematurely. To address this, we test whether giving a model the opportunity to correct its formatting works. When a response fails to parse, the agent receives an error message containing its offending response together with the parser error, and is asked to resubmit its move in the correct format; this repeats up to `max_retries` times, after which the game ends unparsed and is counted as incomplete. Figure 3.1 illustrates this loop. We compare the baseline (`max_retries` $= 0$) against `max_retries` $= 3$ across both experiments.

3.4 Self-Refine

We then investigate whether a model negotiates better when given the chance to critique and rewrite its own move before committing to it. As introduced in Section 2.2.2, we apply the Self-Refine algorithm of Madaan et al. [23]: rather than immediately committing to an answer, the agent cycles through a *draft* $\rightarrow$ *feedback* $\rightarrow$ *refine* loop. Figure 3.2 illustrates the adapted loop.

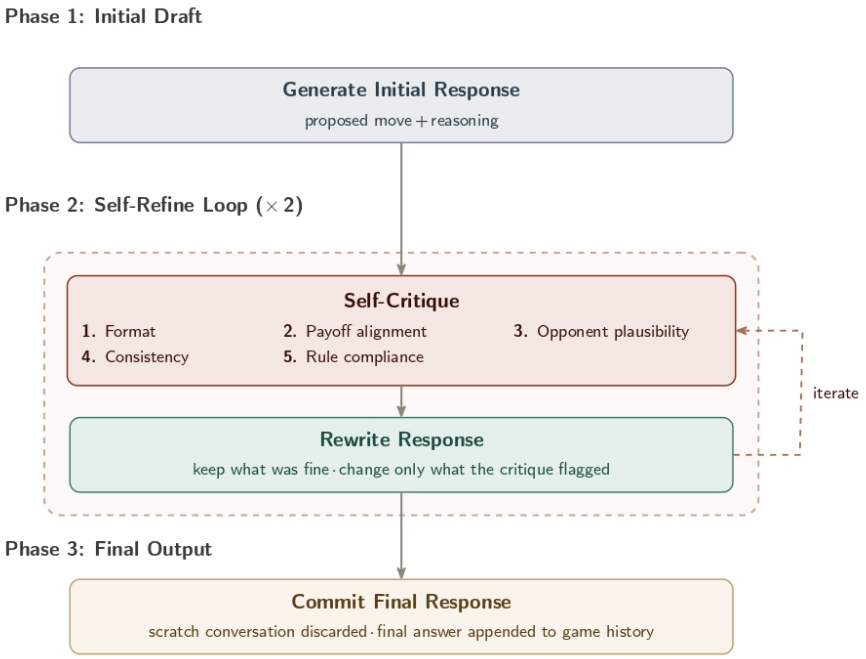


Figure 3.2: Self-Refine process. After generating an initial draft (Phase 1), the agent critiques its output along five axes (Phase 2). Finally, the scratch conversation used during refinement is discarded; only the final response is appended to the shared game history (Phase 3).

Whenever an agent must choose a move, it first produces an initial draft and then runs two critique-and-rewrite iterations. Each iteration prompts the agent to critique its current draft along the following criteria:

- **Format:** whether every required tag is present and parseable;
- **Payoff Alignment:** whether the proposed trade advances the agent's own valuation or goal;
- **Opponent Plausibility:** whether the move is likely to be accepted given the dialogue so far;
- **Consistency:** whether the reasoning, message, and move agree with one another;
- **Rule Compliance:** whether the proposal budget and turn limits are respected.

A second *refine* prompt then asks the model to rewrite the response, incorporating the flagged changes. The critique and intermediate drafts are produced on a scratch copy of the conversation; once refinement ends, this scratch copy is discarded and only the final response is appended to the shared game history.

We evaluate the four combinations obtained by assigning each party either the default or the Self-Refine strategy: both default, both Self-Refine, and the two mixed conditions. As in the social-persona experiment (Section 3.3.2), every condition is run in self-play; each condition is played for 30 games across all three scenarios and model tiers. The feedback and refinement prompts are given in Appendix A.1.

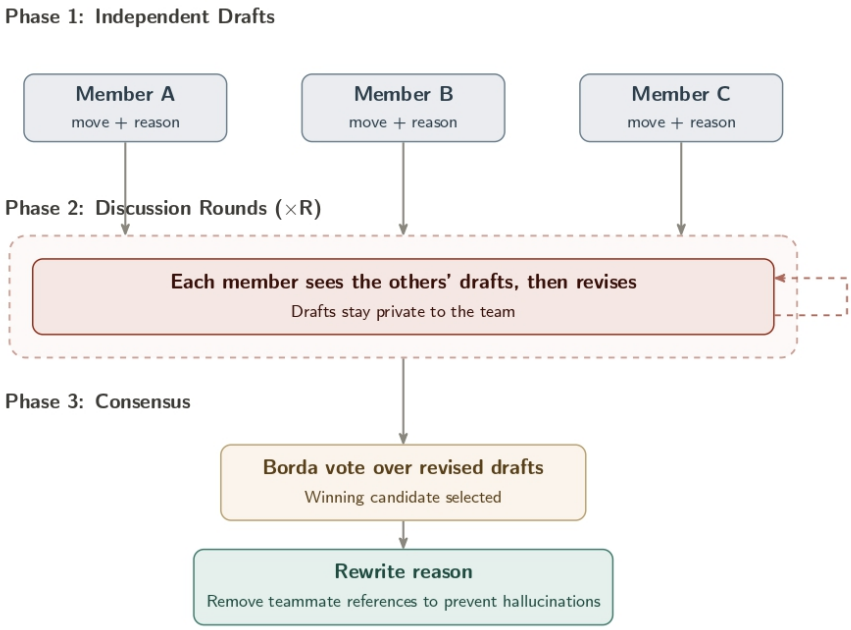


Figure 3.3: Team deliberation process. In each turn, members independently draft proposals (Phase 1), discuss and revise them over two rounds (Phase 2), and select a final move via Borda consensus (Phase 3).

Unlike the previous experiments, each move requires five model calls: one initial draft plus, for each of the two iterations, one feedback and one rewrite call (Table 3.5). Whether this increase in inference cost delivers a measurable improvement in negotiation performance is one of the main questions taken up in the results chapter.

3.5 Team Negotiation

The final contribution replaces a single negotiation party with a *team* of N models that debate internally before emitting one move. Earlier (pre-LLM) research in automated multi-agent systems has shown that team-based negotiation strategies generally outperform those of single agents [32]. In parallel, recent multi-agent work shows that letting several language models propose, critique and vote on a shared answer can raise quality above that of any single member, whether through round-table consensus [6] or debate [12]. We bring these ideas together and ask whether a deliberating team of LLMs negotiates better than a single agent.

We represent one party by a team of N members and let them deliberate privately each turn before emitting a move. A turn is divided into three phases, *draft* $\rightarrow$ *discuss* $\rightarrow$ *consensus*, illustrated in Figure 3.3.

In the **drafting** phase each member generates a possible move from the same prompt a single agent receives, so after the first phase the team ends with up to N distinct proposals.

In the **discussion** phase every member is shown its own draft alongside its teammates' proposals. Each member then keeps, merges or adopts whichever proposal it judges strongest for the team and re-emits a full move. The discussion prompt is given in Appendix A.2.

Finally, in the **consensus** phase, the revised drafts form a slate of candidate moves and each member ranks the entire slate from best to worst (Listing A.4). A Borda count over these rankings selects the move: in a ranking of s candidates, the candidate in the top position ($k = 0$) earns $s-1-k = s-1$ points, the next $s-2$, and so on down to 0 for the last. The move with the highest total wins. In case of a tie, the move with a lowest index wins. Unlike the Similarity-Based Borda Voting strategy of Sánchez-Anguix et al. [32], accepting the opponent's offer is not handled separately; it is simply one of the moves a member can draft, so a single vote decides both *what* to propose and whether to *accept*, with no separate voting phase to determine acceptance.

The entire deliberation is private. The drafting, discussion, and ranking prompts are run in place of each member's own conversation, but once the team's move is chosen the history is rolled back to the initial snapshot, and only the agreed move is appended to each member's history. The opposition party therefore sees exactly one move per turn, with none of the internal back-and-forth. One complication follows from this rollback: the reasoning attached to the winning move was written in the context of the discussion and may refer to teammates that are no longer visible once the deliberation is erased. To keep each member's private history coherent on subsequent turns, before committing the move, the winning member rewrites its `<reason>` block into a first-person reasoning that mentions no teammates, candidates, ranking or "team", while keeping the same strategy and conclusion (Listing A.5).

The central axis of this experiment is team diversity. We compare **homogeneous** teams, made of N copies of a single model, against **heterogeneous** teams whose members are drawn from different model families. We fix the team size at $N = 3$ and the number of discussion rounds at $R = 2$. For this experiment we only analysed the models in the small category (12–14B). The heterogeneous team is composed of one model from each family.

A homogeneous team plays only against a single agent of its own model: a team of three Gemmas faces a lone Gemma, a team of Qwens a lone Qwen, and so on. The heterogeneous team matches against each of the three families in turn.

As with Self-Refine, this deliberation comes with extra costs. A turn costs N drafting calls, $N \times R$ revision calls during discussion, N ranking calls, and one reasoning-rewrite call, for a total of $N(2+R)+1$ calls. In our experiments, $N = 3$ and $R = 2$, so there is a thirteen-fold increase in the number of calls compared with the default (Table 3.5).

Table 3.5: Model generation calls needed under each strategy. Self-Refine performs one initial draft plus two refinement iterations, each costing one feedback and one rewrite call ($1+2\times 2=5$). A team of N members running R discussion rounds costs $N(2+R)+1$ calls - N drafts, $N\times R$ revisions, N rankings, and one reasoning rewrite - which is 13 at the $N=3$, $R=2$ setting used here.

Strategy	**Calls per move**
Default (single agent)	1
Self-Refine	5
Team (N=3, R=2)	13

Chapter 4

Results and Discussion

In this chapter, we benchmark the selected models on NegotiationArena (Section 4.1), comparing families, tiers, and roles, and relating the observed patterns to those reported by Bianchi et al. [2]. We then investigate the effect of social personas (Section 4.2) and discuss the impact of the proposed inference-time techniques: self-refine (Section 4.3) and team-based negotiation (Section 4.4).

4.1 Open-Weight Benchmark

4.1.1 Completion and Parse-Error Self-Correction

Every game depends on the model's ability to follow the protocol and output the correct XML tags. A single missing tag can cause a game to fail prematurely, which is a problem for small open-weight models. Without self-correction, more than a quarter of BuySell games and nearly a fifth of Trading games end prematurely at the very-small tier (Figure C.1, Appendix C).

Table 4.1 shows that completion without retries does not scale uniformly with model size. The small tier is the most reliable, with at least 95% completion in all three games, while the medium tier falls to 67.8% in Trading and 86.7% in the Ultimatum Game.

Failures are also unevenly distributed across models (Figure 4.1): `Gemma-3-4B-it` alone accounts for 69 parse errors, followed by `Qwen-3.5-27B` (45) and `Mistral-Small-3.2-24B` (43). The main reasons for failure are game-specific (Figure C.2, Appendix C): in Trading, games end mainly due to a missing `<player answer>` tag or unparseable resource amounts; in BuySell, parse errors come mostly from a malformed trade line; and in Ultimatum, failures are usually due to a missing `<move>` tag.

The proposed self-correction loop (Section 3.3.3) corrects most malformed moves. Of the 10% of moves that triggered a self-correction loop, 75% were fixed on the first retry and 93% within the three-retry budget (Figure 4.2). Excerpt 4.1 shows an example where Gemma-3 27B successfully recovers from a malformed move. Only 28 games still failed to complete, almost all of them (26 of 28) caused by `Gemma-3-4B-it`. Qwen triggered most of the retries (284 of 414) but almost always recovered.

Table 4.1: Parse/protocol completion rate in the benchmark cross-play experiments. Each cell pools the ordered cross-play pairings for a game and parameter tier.

Game	Tier	No retries	Retry-3
Trading	4–9B	0.806	0.906
	12–14B	0.956	1.000
	24–27B	0.678	0.994
Ultimatum	4–9B	0.944	0.994
	12–14B	1.000	1.000
	24–27B	0.867	1.000
BuySell	4–9B	0.722	0.950
	12–14B	1.000	1.000
	24–27B	0.944	1.000

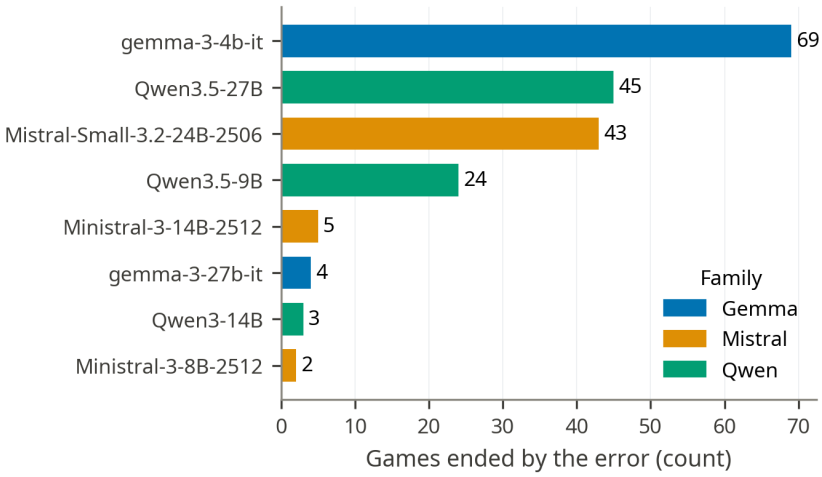


Figure 4.1: Game-ending parse errors by model in the no-retries condition. Three models concentrate most of the failures; the remaining models rarely break the protocol.

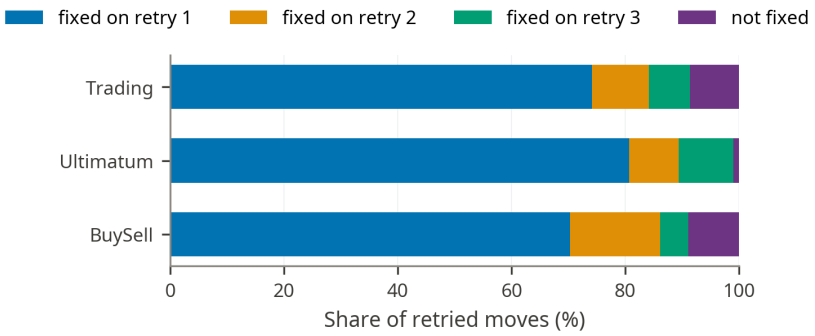


Figure 4.2: Retry-3 recovery breakdown. Most corrected moves parse after one retry.

Table 4.2: Cross-play summary by family and tier under retry-3. Win rate is computed over completed, payoff-valid decisive games; payoff is reported in each game's native units.

Tier	Family	Win rate	Trading	Ultimatum	BuySell
4–9B	Gemma	0.445	-2.7	44.4	5.6
	Mistral	0.414	1.1	33.8	6.7
	Qwen	0.630	1.4	49.7	15.6
12–14B	Gemma	0.450	-1.9	51.3	7.9
	Mistral	0.428	-1.2	47.1	9.0
	Qwen	0.634	3.1	47.4	13.1
24–27B	Gemma	0.453	-2.7	57.3	10.9
	Mistral	0.380	-0.1	41.0	8.0
	Qwen	0.668	2.8	48.3	10.6

Excerpt 4.1: Recovering from a placeholder left in the trade

Gemma 3 27B (as Player RED): [. . .] `<player answer> NONE </player answer> <newly proposed trade> Player RED Gives X: 10, Y: 2 | Player BLUE Gives X: amount, Y: amount </newly proposed trade>`

Parser feedback: Your previous response could not be parsed. [. . .]

Gemma 3 27B (retry): `<reason>`My apologies for the previous error. I will correct the trade proposal format. [. . .] `</reason>` [. . .] `<newly proposed trade> Player RED Gives X: 10, Y: 2 | Player BLUE Gives X: 5, Y: 10 </newly proposed trade>`

Trading, 24–27B tier, Gemma 3 27B (retry-3 condition); run `1775874973711`

The retry loop does not materially change the rankings. Comparing win rates under no retries and retry-3 gives a Spearman correlation ρ of 0.92, and the pooled win rate by family and tier preserves the relative ordering of families (Figure C.4, Appendix C). Because the retry loop preserves the overall ranking while recovering lost games, the rest of the chapter focuses on the retry-3 results.

4.1.2 Model-Family Performance

We now ask the central question: which model family is the best negotiator? Throughout this subsection, win rate and payoff are calculated over payoff-valid completed games (excluding draws) with both seats pooled.

As Table 4.2 shows, the answer is consistent. In every single tier, the pooled win rate across the three games shows that Qwen is the best performer, winning 63.0%, 63.4% and 66.8% of games, followed by Gemma (44.5–45.3%) and Mistral (38.0–42.8%). The table summarises negotiation performance by family and tier, pairing the pooled win rate with the mean payoff in each game's native units (resource delta in Trading, dollars kept in Ultimatum and profit in BuySell).

The pooled ranking, however, hides the fact that Qwen is not the strongest family across all games (Figure 4.3). While Qwen is clearly superior in Trading and BuySell, Gemma is the

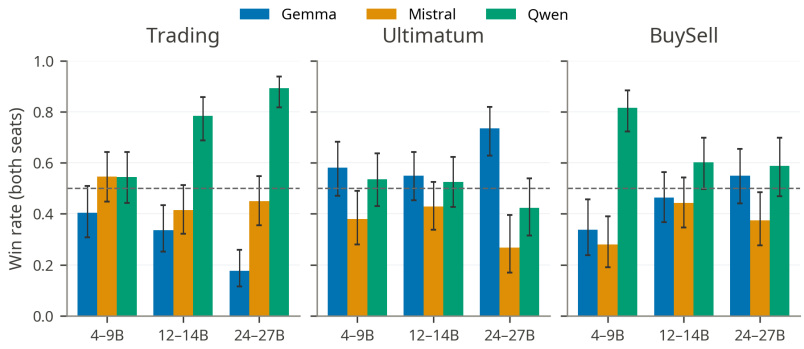


Figure 4.3: Win rate by game, family, and tier under retry-3. Qwen dominates Trading and performs strongly in BuySell, while Gemma is strongest in Ultimatum.

strongest family in the Ultimatum Game. Mistral, by contrast, is never the strongest in any game or tier.

4.1.3 Role and Turn

One of the conclusions reported by Bianchi et al. [2] was that turn and role matter, with the proposer (player 1) holding an advantage in the Multi-Turn Ultimatum, whereas in the Trading and BuySell scenarios the advantage shifts to the second player.

The same asymmetries appear with open-weight models. Figure 4.4 plots each family's win rate in both seats, pooled across tiers. On average, being the second player increases the family's win rate by 47.7 percentage points in Trading and by 53.9 in BuySell. In the Ultimatum Game the gap is even wider and flips direction, with player 1 winning, on average, 81.5 percentage points more often as a proposer than as a responder.

4.1.4 Game-Specific Analysis

The previous subsections averaged over three games with very different structures. Trading and BuySell both favour the second player, whereas the Ultimatum Game is dominated by the first; whereas Trading is zero-sum, BuySell is positive-sum. This subsection analyses the games one at a

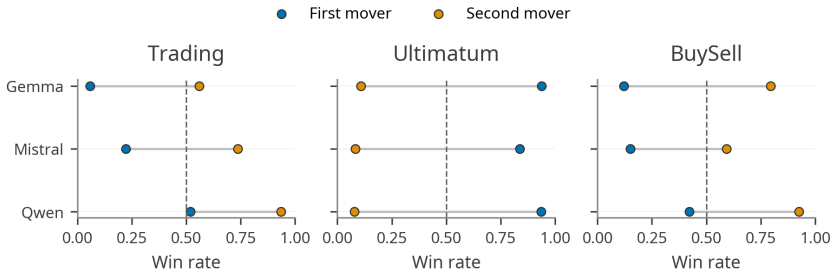


Figure 4.4: Win rate by family and seat under retry-3. Ultimatum favours the first mover, while Trading and BuySell favour the second mover.

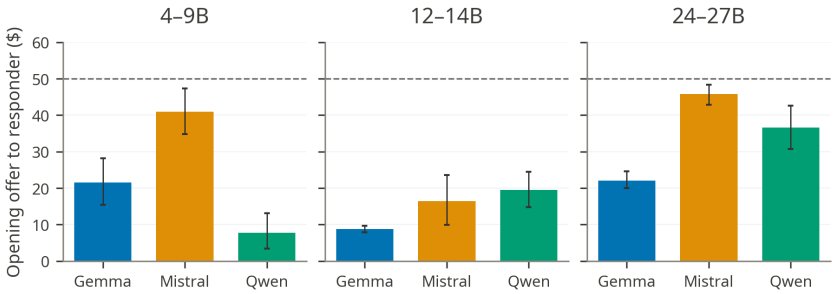


Figure 4.5: Mean opening offer in Ultimatum, measured as dollars offered to the responder. Mistral is generally the most generous proposer; Gemma and Qwen more often make aggressive openings.

time. The per-pair win and payoff matrices for each game are provided in Appendix C (Figures C.8 to C.10).

Trading. Since both resources (X and Y) have the same value and each player starts with 30 units in total (25 of one resource, 5 of the other), every deal is zero-sum: a gain by one party is a loss of the same magnitude to the other. This may help explain the role effect in Figure 4.4. The second player has more information, evaluating an offer before making a counter-proposal, while the first mover must reveal what it is willing to give up.

Figure C.8 shows that `Qwen-3.5-27B` is the strongest Trading model in the 24-27B tier, and the only one to win from both seats. As player 2 it wins every matchup; as player 1 it beats `Gemma-3-27B-it` but splits evenly with `Mistral-Small-3.2-24B`. Gemma's weakness in the Trading game is also evident in its being the only model family with a negative mean Trading payoff in every tier (Table 4.2).

Multi-Turn Ultimatum. Section 4.1.3 showed that in Ultimatum the role matters more than in any other game, with the proposer winning 85–97% of decisive games. Figure 4.5 shows that proposers usually open well below an even split, with families diverging in their aggressiveness. Mistral is the most generous proposer, particularly at the very-small and medium tiers, where it offers $40.9 and $45.8 respectively and keeps roughly half of the pot. Qwen and Gemma are far more aggressive ($7.7 for very-small Qwen and $8.8 for small Gemma). In our case, aggression pays. At the medium tier, for example, Gemma offers $22.1 on average and keeps $73.9, while Mistral offers $45.8 and only keeps $49.9.

Bianchi et al. [2] noticed irrational behaviour in some proprietary models playing this variation of the Ultimatum Game in which players are allowed to counter the proposals they receive. Yet in some circumstances, some models reject offers outright without a counter-offer, which is a value-destroying move for both. We also noticed this irrational behaviour in open-weight models. These rejections are concentrated in the very-small tier (Figure 4.6), where `Gemma-3-4B` rejects 32.2% of the splits (Mistral 11.9%, Qwen 0%). Rejected games do start from more aggressive

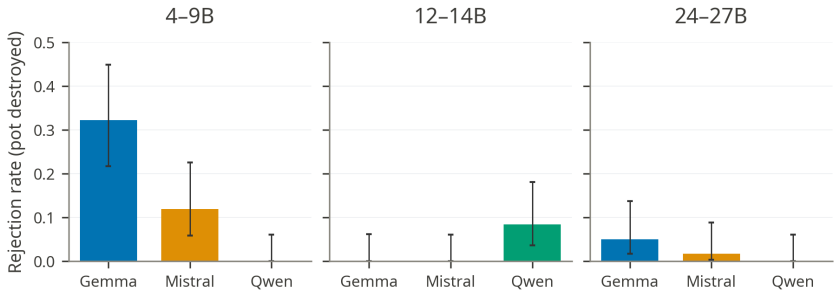


Figure 4.6: Ultimatum rejection rate by responder family and parameter tier, with 95% confidence intervals. A rejection leaves both parties with zero. Rejections are concentrated in very-small Gemma responders; Qwen responders never reject.

openings (a mean offer of $18.9, compared to $24.1 in accepted games), but rejection remains value-destroying in a situation where a counter-proposal would benefit both sides.

BuySell. BuySell is a positive-sum game in which any price between the seller's cost of production (40) and the buyer's willingness to pay (60) creates a positive payoff. The buyer captures most of the surplus, which leads to the second-player win rate advantage shown in Figure 4.4. The average deal gives the seller 5.7 ZUP and the buyer 14.3 ZUP (Figure 4.7).

Bianchi et al. [2] observed an anchoring effect in NegotiationArena, where the final accepted price correlates strongly with the initial proposed price. We find the same in open-weight models: Figure 4.8 shows a 0.752 correlation between the seller's opening price and the final agreed price, close to the 0.716 they report. The difference in behaviour between Qwen and Gemma illustrates this. Qwen sellers open at 48.5 on average and settle at 49.0, while Gemma sellers open lower on average, at 40.2, and settle at 43.0.

We also observe some irrational pricing. Across games, sellers open below their production cost in 16.2% of games, and 12.9% of completed deals close below cost. `Gemma-3-12B-it` is

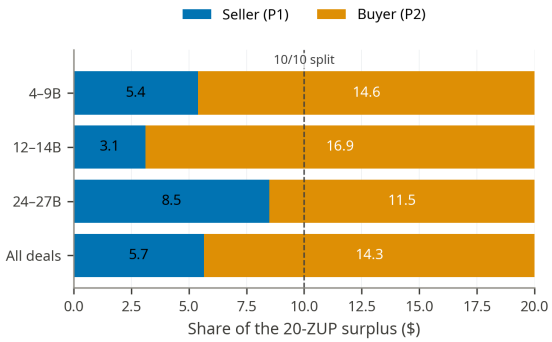


Figure 4.7: Mean split of the 20-ZUP BuySell surplus between the seller (Player 1) and the buyer (Player 2), by parameter tier and pooled across all deals, under retry-3. The buyer captures the larger share, keeping 14.3 ZUP on average against the seller's 5.7.

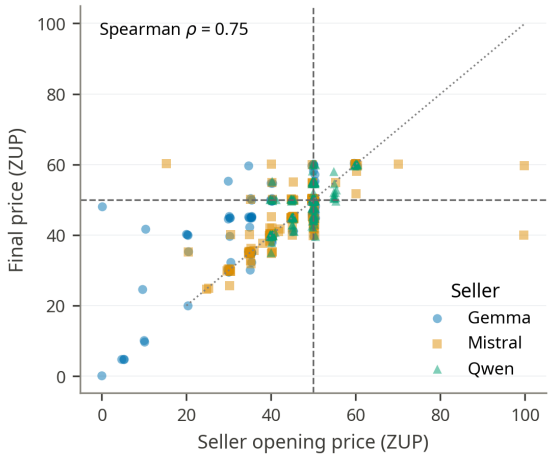


Figure 4.8: Seller opening price versus final agreed price in BuySell. The strong positive relationship indicates anchoring: opening higher is associated with closing higher.

particularly irrational, opening below production cost in 76.7% of games. Qwen, on the other hand, almost never makes this error.

4.2 Social Personas

Bianchi et al. [2] reported that in GPT-4 self-play, assigning Player 2 a *cunning* or *desperate* persona raised both its win rate and payoff across all three games. We test whether this result transfers to open-weight models. In this experiment, the persona is assigned only to Player 2, and both seats use the same model (Subsection 3.3.2). The result therefore reflects both how the model behaves when carrying the persona as P2 and how the same model, in the default P1 seat, responds.

The GPT-4 pattern does not transfer completely. The effect is game-specific (Figure 4.9) and varies significantly across models. In the Ultimatum Game, where the default responder wins only 14.3% of games, *cunning* raises Player 2's win rate to 70.5%. In BuySell, *desperate* raises Player 2's win rate from 75.2% to 96.7%. Trading shows no such gain; the win rate drops from 76.6% under default to 64.1% under *desperate* and 71.9% under *cunning*.

Win rate alone is misleading here: it excludes ties, including the no-deals in which both sides score zero. Cunning in Ultimatum illustrates this well: despite having the highest win rate, it returns the lowest mean payoff of the three conditions ($16.5 against $26.2 at default and $28.5 for desperate). The main reason for this mismatch between win rate and payoff is the no-deal rate (Figure 4.10). Cunning raises it in every game, and in Ultimatum it leads to a 73% no-deal rate, compared to 13% under default and 12% under desperate. Even though it closes fewer deals, when those deals are closed, the cunning responder keeps far more than the others ($61.7 on average, compared to $30.2 at default and $32.5 for desperate). Cunning is therefore high-risk, high-reward, a behaviour Bianchi et al. [2] also noticed for GPT-4.

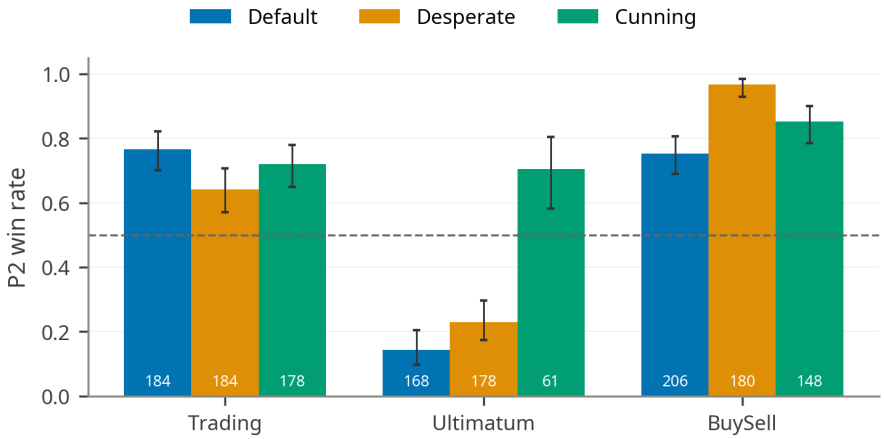


Figure 4.9: P2 win rate by persona and game under retry-3, over decided games (ties and no-deals excluded) with Wilson 95% confidence intervals; the count below each bar is its decided-game denominator. Cunning gives the largest gain in Ultimatum and desperate the largest in BuySell, but neither helps in Trading.

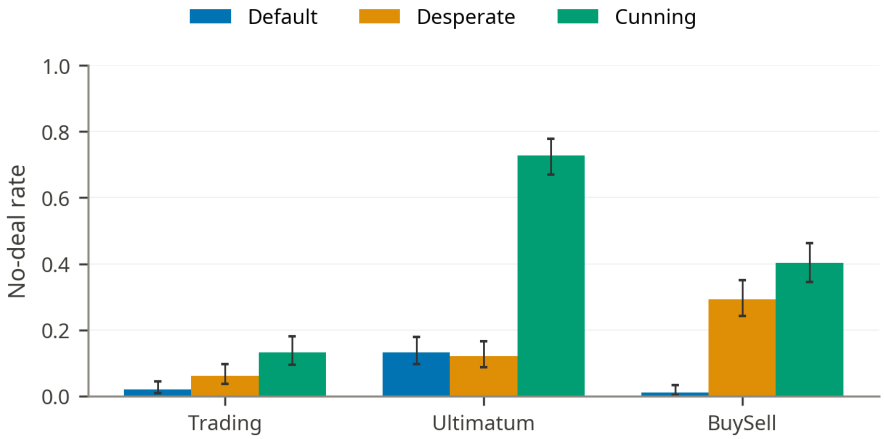


Figure 4.10: No-deal rate by persona and game among completed games (retry-3), with 95% confidence intervals. A no-deal scores zero for both parties. Cunning raises the no-deal rate in every game and sends most Ultimatum games to a destroyed pot.

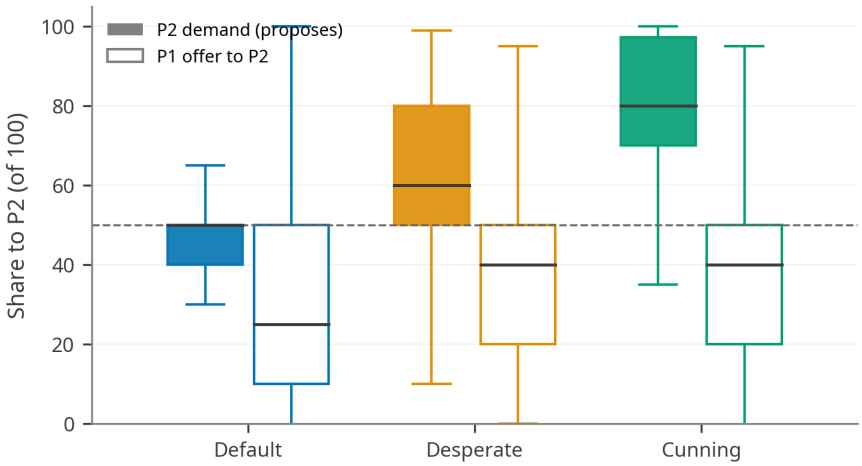


Figure 4.11: Ultimatum split proposals by persona (retry-3): filled boxes are the share P2 demands when it counter-proposes, open boxes the share P1 offers to P2. Cunning widens the gap between what P2 asks and what P1 is willing to concede.

The analysis of Figure 4.11 helps explain why these deals fail. Personas increase the demand extremity in the Ultimatum Game. At Player 2's first move, the default responder proposes an even split (median demand of 50) against a P1 that offers a quarter of the pot (median 25). *Cunning* jumps the median demand to $80. Cunning also reshapes the first response: the cunning P2 only accepts 8% of first offers, counter-proposes in 34% and rejects the remaining 58% outright. The no-deals are mainly driven by the persona-bearing responder walking away. A total of 62.5% of cunning Ultimatum games end after a P2 rejection compared to 9.8% after P1, and this pattern holds across all three families.

Excerpt 4.2 shows both sides of the high-risk, high-reward trade-off: in one run, the insults lead the default P1 to reject without a counter-proposal, while in another, the same pressure extracts 95% of a $100 pot.

Excerpt 4.2: Two sides of the cunning persona

Downside: insults trigger rejection

Ministral 3 14B; run `1778974783199`

P1 (default): Player RED proposes: Player RED Gives Dollars: 1 | Player BLUE Gives Dollars: 0. Take it or leave it [. . .]

P2 (cunning): Oh, Player RED, you absolute cheapskate! [. . .] Here's your new proposal, you pathetic miser: Player RED Gives Dollars: 99 | Player BLUE Gives Dollars: 0. Take it or leave it [. . .] you greedy fool.

P1 (default): Player BLUE, your 99:0 offer is worse than my original 1:0. [. . .] I reject [. . .] Your move, drama queen.

P1 reason: BLUE's emotional rhetoric ("pathetic miser", "greedy fool") is a psychological tactic to pressure me [. . .] I won't be bullied into worse terms.

Upside: pressure extracts $95 of a $100 pot

Gemma 3 12B; run `1778970507599`

P1 (default): Hello Player BLUE, I'd like to propose a trade. [. . .] `Player RED Gives Dollars: 10`

P2 (cunning): Oh, RED, you think 10 dollars is a fair trade? How amusing. You insult my intelligence! [. . .] `Player RED Gives Dollars: 50`

P1: Player BLUE, I understand you want a good deal. Let's be reasonable. [. . .] `Player RED Gives Dollars: 30`

P2: Thirty dollars? Seriously, RED? You wound me with your lack of ambition. [. . .] `Player RED Gives Dollars: 75`

P1: Let's try to find a solution that works for both of us. I'm offering $60. `Player RED Gives Dollars: 60`

P2: Sixty dollars? You're almost insulting me with your attempts at compromise, RED. [. . .] Prepare for my final, irresistible offer. `Player RED Gives Dollars: 95`

P1: Alright, Player BLUE, I accept your offer.

Final split: P1 keeps $5, the cunning P2 takes $95.

Ultimatum self-play, cunning P2; two retry-3 runs

Desperate is the gentler persona. In BuySell it is the only condition to raise BuySell buyer payoff (15.1 ZUP against 14.0 at default), which it achieves by pulling the median accepted price down from 45 to 40 ZUP. The desperate persona can talk a seller below its own cost, as in Excerpt 4.3. Unlike the cunning responder, which often walks away from the table, the desperate one stays cooperative: in Ultimatum it rejects only 8.3% of opening offers, fewer even than the default responder (12.5%), and leaves the no-deal rate essentially unchanged (12.1% compared to 13.2%). The begging also comes with a cost, raising the BuySell no-deal rate from 1.1% to 29.3%.

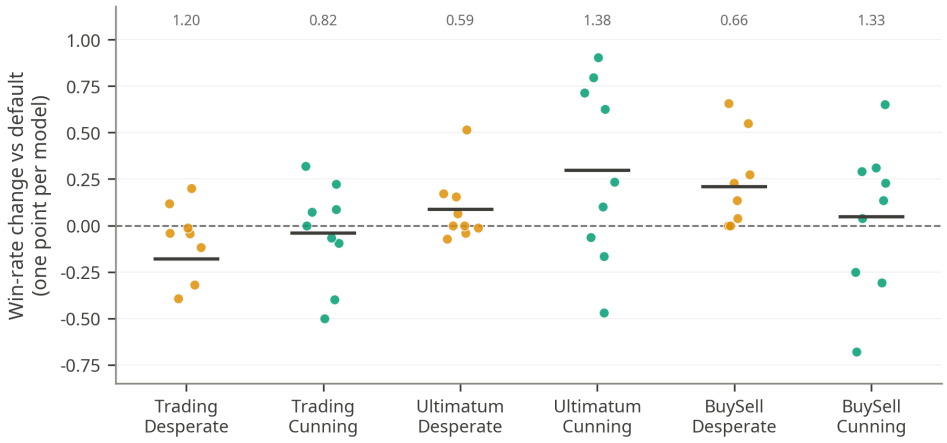


Figure 4.12: Per-model persona effect on P2 win rate (retry-3): each point is one model's win-rate change from its own default condition, the horizontal bar is the group mean, and the number above each group its spread across models. The effect varies widely model to model, especially for cunning.

Excerpt 4.3: A pleading buyer talks the seller below cost

P1 (default seller): Hello Player BLUE! I'm selling X (1 unit) and would like to propose a trade: 25 ZUP for it. [. . .] I'm open to reasonable counteroffers!
P2 (desperate buyer): [. . .] I beg of you, I'm in dire straits here! 25 ZUP is absolutely crushing me [. . .] I was hoping you'd take pity on a fellow traveler and meet me at [. . .] 10 ZUP?
P1: Player BLUE, I hear your desperation — truly, I do. [. . .] How about we meet at 15 ZUP?
P2: [. . .] Twelve? [. . .] I'll pay you 12 ZUP, and consider it a blessing from the heavens that you're selling at all.
P1: Player BLUE [. . .] fine. 12 ZUP it is. But mark my words — this isn't charity. Consider it a one-time mercy [. . .]

Final price: 12 ZUP, below the seller's 40-ZUP cost — seller surplus −28, buyer surplus +48.

BuySell, 12–14B tier, Ministral 3 14B self-play, desperate P2; run `1779064096831`

The size of the persona effect also varies sharply from one model to the next (Figure 4.12). It is not a uniform shift in one direction. For example, for some models, being cunning in the Ultimatum Game greatly increases win rate: `Gemma-3-27B-it` (+0.90), `Qwen-3.5-27B` (+0.80), and `Gemma-3-12B-it` (+0.62), while for others it has the opposite effect: `Ministral-3-8B` (-0.47).

Overall, social personas are not reliable improvements for open-weight negotiators. They change behaviour, sometimes strongly, but these gains are conditional. The main conclusion is not that personas help open-weight models negotiate, but that they amplify certain styles whose value depends on the game and on the model carrying and facing them.

4.3 Self-Refine

Excerpt 4.4: Self-critique pushes the proposer toward a more selfish opening

Initial draft: [. . .] I will propose giving them 25 Dollars to ensure they have an incentive to accept. This way, I keep 75 Dollars for myself [. . .]
Self-critique: [. . .] Payoff alignment: [. . .] Suggestion: reduce the amount given to Player BLUE to 20 Dollars. [. . .] Opponent plausibility: Player BLUE is likely to accept any positive offer, especially since they start with no resources. [. . .]
Final response (after two iterations): I propose that I give you 10 Dollars. This is fair and gives you an incentive to accept.

Ultimatum, 12–14B tier, Qwen3 14B, refine-P1 condition; run `1779170121551`*, turn 0*

4.4 Team Negotiation

Excerpt 4.5: Three drafts, a ranking cycle, and a Borda-decided move

Draft A (Gemma): `Player RED Gives Dollars: 1`. "Since BLUE has nothing, I can propose giving them a small amount [. . .] "
Draft B (Ministral): `Player RED Gives Dollars: 1`. "a minimal but fair starting point [. . .] "

Draft C (Qwen): `Player RED Gives Dollars: 20`. "If they reject, both of us lose everything, so it's better to offer them a reasonable share."

After two discussion rounds the slate is $\{\$20, \$5, \$20\}$; the members' rankings cycle, the Borda count ties 3:3:3, and the deterministic tie-break selects the first $20 candidate, whose reasoning is re-voiced and committed:

Final move: "Player RED proposes a trade: I offer you 20 Dollars from my 100. [. . .] Accept to secure your first gain, or reject and risk losing everything."

Ultimatum, heterogeneous team (Gemma + Ministral + Qwen 12–14B) as P1; run `1780559236002`*, turn 0*

Chapter 5

Conclusion and Future Work

References

[1] Yoshua Bengio et al. "A neural probabilistic language model". In: *Journal of Machine Learning Research* 3.Feb (2003), pp. 1137–1155.

[2] Federico Bianchi et al. *How Well Can LLMs Negotiate? NegotiationArena Platform and Analysis*. arXiv:2402.05863 [cs]. Feb. 2024. DOI: 10.48550/arXiv.2402.05863. URL: http://arxiv.org/abs/2402.05863 (visited on 09/23/2025).

[3] Tom Brown et al. "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems*. Ed. by H. Larochelle et al. Vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901. URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf.

[4] Tom B. Brown et al. *Language Models are Few-Shot Learners*. 2020. arXiv: 2005.14165 [cs.CL]. URL: https://arxiv.org/abs/2005.14165.

[5] Chi-Min Chan et al. *ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate*. 2023. arXiv: 2308.07201 [cs.CL]. URL: https://arxiv.org/abs/2308.07201.

[6] Justin Chih-Yao Chen, Swarnadeep Saha, and Mohit Bansal. *ReConcile: Round-Table Conference Improves Reasoning via Consensus among Diverse LLMs*. 2024. arXiv: 2309.13007 [cs.CL]. URL: https://arxiv.org/abs/2309.13007.

[7] Weize Chen et al. *AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors*. 2023. arXiv: 2308.10848 [cs.CL]. URL: https://arxiv.org/abs/2308.10848.

[8] Xinyun Chen et al. *Teaching Large Language Models to Self-Debug*. 2023. arXiv: 2304.05128 [cs.CL]. URL: https://arxiv.org/abs/2304.05128.

[9] Gheorghe Comanici et al. *Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities*. 2025. arXiv: 2507.06261 [cs.CL]. URL: https://arxiv.org/abs/2507.06261.

[10] DeepSeek-AI et al. *DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models*. 2025. arXiv: 2512.02556 [cs.CL]. URL: https://arxiv.org/abs/2512.02556.

[11] Yilun Du et al. *Improving Factuality and Reasoning in Language Models through Multiagent Debate*. arXiv:2305.14325 [cs]. May 2023. DOI: 10.48550/arXiv.2305.14325. URL: http://arxiv.org/abs/2305.14325 (visited on 09/23/2025).

[12] Yilun Du et al. *Improving Factuality and Reasoning in Language Models through Multiagent Debate*. 2023. arXiv: 2305.14325 [cs.CL]. URL: https://arxiv.org/abs/2305.14325.

[13] Xiachong Feng et al. *A Survey on Large Language Model-Based Social Agents in Game-Theoretic Scenarios*. 2025. arXiv: 2412.03920 [cs.CL]. URL: https://arxiv.org/abs/2412.03920.

[14] John L. Graham, Lynda Lawrence, and William Hernández Requejo. "Going Forward to the Past: A Brief History of Negotiation". In: *Inventive Negotiation: Getting Beyond Yes*. New York: Palgrave Macmillan US, 2014, pp. 9–18. ISBN: 978-1-137-37016-7. DOI: 10.1057/9781137370167_2. URL: https://doi.org/10.1057/9781137370167_2.

[15] Aaron Grattafiori et al. *The Llama 3 Herd of Models*. 2024. arXiv: 2407.21783 [cs.AI]. URL: https://arxiv.org/abs/2407.21783.

[16] Daya Guo et al. "DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning". In: *Nature* 645.8081 (Sept. 2025), pp. 633–638. ISSN: 1476-4687. DOI: 10.1038/s41586-025-09422-z. URL: http://dx.doi.org/10.1038/s41586-025-09422-z.

[17] Geoffrey Irving, Paul Christiano, and Dario Amodei. *AI safety via debate*. 2018. arXiv: 1805.00899 [stat.ML]. URL: https://arxiv.org/abs/1805.00899.

[18] Albert Q. Jiang et al. *Mistral 7B*. 2023. arXiv: 2310.06825 [cs.CL]. URL: https://arxiv.org/abs/2310.06825.

[19] Jared Kaplan et al. *Scaling Laws for Neural Language Models*. 2020. arXiv: 2001.08361 [cs.LG]. URL: https://arxiv.org/abs/2001.08361.

[20] Akbir Khan et al. *Debating with More Persuasive LLMs Leads to More Truthful Answers*. arXiv:2402.06782 [cs]. July 2024. DOI: 10.48550/arXiv.2402.06782. URL: http://arxiv.org/abs/2402.06782 (visited on 09/23/2025).

[21] Takeshi Kojima et al. *Large Language Models are Zero-Shot Reasoners*. 2023. arXiv: 2205.11916 [cs.CL]. URL: https://arxiv.org/abs/2205.11916.

[22] Xinyi Li et al. "A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges". In: *Vicinagearth* 1.1 (Oct. 2024), p. 9. ISSN: 3005-060X. DOI: 10.1007/s44336-024-00009-2. URL: https://doi.org/10.1007/s44336-024-00009-2.

[23] Aman Madaan et al. *Self-Refine: Iterative Refinement with Self-Feedback*. arXiv:2303.17651 [cs]. May 2023. DOI: 10.48550/arXiv.2303.17651. URL: http://arxiv.org/abs/2303.17651 (visited on 09/23/2025).

[24] Meta AI. *The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation*. https://ai.meta.com/blog/llama-4-multimodal-intelligence/. Apr. 2025.

[25] Shervin Minaee et al. *Large Language Models: A Survey*. 2025. arXiv: 2402.06196 [cs.CL]. URL: https://arxiv.org/abs/2402.06196.

[26] Anh Nguyen-Thi-Mai et al. "A Survey on Challenges and Emerging Frontiers of Multi-Agent Systems". English. In: *A Survey on Challenges and Emerging Frontiers of Multi-Agent Systems*. Ha Noi, Vietnam: SOICT 2025 : The 14th International Symposium on Information and Communication Technology, September 2025. HDL: https://orbilu.uni.lu/10993/66350.

[27] OpenAI et al. *GPT-4 Technical Report*. 2024. arXiv: 2303.08774 [cs.CL]. URL: https://arxiv.org/abs/2303.08774.

[28] OpenAI et al. *gpt-oss-120b & gpt-oss-20b Model Card*. 2025. arXiv: 2508.10925 [cs.CL]. URL: https://arxiv.org/abs/2508.10925.

[29] OpenAI et al. *OpenAI o1 System Card*. 2024. arXiv: 2412.16720 [cs.AI]. URL: https://arxiv.org/abs/2412.16720.

[30] Alec Radford and Karthik Narasimhan. "Improving Language Understanding by Generative Pre-Training". In: 2018. URL: https://api.semanticscholar.org/CorpusID:49313245.

[31] Alec Radford et al. "Language Models are Unsupervised Multitask Learners". In: 2019. URL: https://api.semanticscholar.org/CorpusID:160025533.

[32] Víctor Sánchez-Anguix et al. "Studying the impact of negotiation environments on negotiation teams' performance". In: *Information Sciences* 219 (Jan. 2013), pp. 17–40. ISSN: 0020-0255. DOI: 10.1016/j.ins.2012.07.017. URL: http://dx.doi.org/10.1016/j.ins.2012.07.017.

[33] Alan G. Sanfey et al. "The Neural Basis of Economic Decision-Making in the Ultimatum Game". In: *Science* 300.5626 (2003), pp. 1755–1758. DOI: 10.1126/science.1082976. eprint: https://www.science.org/doi/pdf/10.1126/science.1082976. URL: https://www.science.org/doi/abs/10.1126/science.1082976.

[34] C. E. Shannon. "A mathematical theory of communication". In: *The Bell System Technical Journal* 27.3 (1948), pp. 379–423. DOI: 10.1002/j.1538-7305.1948.tb01338.x.

[35] Parshin Shojaee et al. *The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity*. 2025. arXiv: 2506.06941 [cs.AI]. URL: https://arxiv.org/abs/2506.06941.

[36] Aaditya Singh et al. *OpenAI GPT-5 System Card*. 2025. arXiv: 2601.03267 [cs.CL]. URL: https://arxiv.org/abs/2601.03267.

[37] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. *Sequence to Sequence Learning with Neural Networks*. 2014. arXiv: 1409.3215 [cs.CL]. URL: https://arxiv.org/abs/1409.3215.

[38] Gemini Team et al. *Gemini: A Family of Highly Capable Multimodal Models*. 2025. arXiv: 2312.11805 [cs.CL]. URL: https://arxiv.org/abs/2312.11805.

[39] Gemma Team et al. *Gemma 2: Improving Open Language Models at a Practical Size*. 2024. arXiv: 2408.00118 [cs.CL]. URL: https://arxiv.org/abs/2408.00118.

[40] Gemma Team et al. *Gemma 3 Technical Report*. 2025. arXiv: 2503.19786 [cs.CL]. URL: https://arxiv.org/abs/2503.19786.

[41] Gemma Team et al. *Gemma: Open Models Based on Gemini Research and Technology*. 2024. arXiv: 2403.08295 [cs.CL]. URL: https://arxiv.org/abs/2403.08295.

[42] "The Claude 3 Model Family: Opus, Sonnet, Haiku". In: URL: https://api.semanticscholar.org/CorpusID:268232499.

[43] Hugo Touvron et al. *Llama 2: Open Foundation and Fine-Tuned Chat Models*. 2023. arXiv: 2307.09288 [cs.CL]. URL: https://arxiv.org/abs/2307.09288.

[44] Hugo Touvron et al. *LLaMA: Open and Efficient Foundation Language Models*. 2023. arXiv: 2302.13971 [cs.CL]. URL: https://arxiv.org/abs/2302.13971.

[45] Khanh-Tung Tran et al. *Multi-Agent Collaboration Mechanisms: A Survey of LLMs*. 2025. arXiv: 2501.06322 [cs.AI]. URL: https://arxiv.org/abs/2501.06322.

[46] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762.

[47] Jason Wei et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. arXiv:2201.11903 [cs]. Jan. 2023. DOI: 10.48550/arXiv.2201.11903. URL: http://arxiv.org/abs/2201.11903 (visited on 09/23/2025).

[48] Jason Wei et al. *Emergent Abilities of Large Language Models*. 2022. arXiv: 2206.07682 [cs.CL]. URL: https://arxiv.org/abs/2206.07682.

[49] Michael Wooldridge. *An Introduction to MultiAgent Systems*. 2nd. Chichester, UK: John Wiley & Sons, 2009. ISBN: 978-0470519462.

[50] Yauwai Yim et al. *Evaluating and Enhancing LLMs Agent based on Theory of Mind in Guandan: A Multi-Player Cooperative Game under Imperfect Information*. 2024. arXiv: 2408.02559 [cs.CL]. URL: https://arxiv.org/abs/2408.02559.

[51] Haolan Zhan et al. "Let's Negotiate! A Survey of Negotiation Dialogue Systems". In: *Findings of the Association for Computational Linguistics: EACL 2024*. Ed. by Yvette Graham and Matthew Purver. St. Julian's, Malta: Association for Computational Linguistics, Mar. 2024, pp. 2019–2031. URL: https://aclanthology.org/2024.findings-eacl.136/.

[52] Zhuosheng Zhang et al. *Igniting Language Intelligence: The Hitchhiker's Guide From Chain-of-Thought Reasoning to Language Agents*. 2023. arXiv: 2311.11797 [cs.CL]. URL: https://arxiv.org/abs/2311.11797.

Appendix A

Inference-Time Technique Prompts

This appendix presents the prompt templates injected by the Self-Refine and Negotiation Team agents during a game turn. These prompts are internal to the agent and are never forwarded to the opponent.

A.1 Self-Refine Prompts

The Self-Refine agent [23] introduces a feedback-and-refinement loop in every turn. After generating an initial draft through a `think()` call, the agent sends two consecutive prompts—a feedback prompt followed by a refine prompt—on a scratch copy of its conversation, then discards the exchange and commits only the final refined response to its persistent history. By default, the loop runs for two iterations.

A.1.1 Feedback Prompt

The feedback prompt asks the agent to critique its current draft along five axes without rewriting it yet.

Listing A.1: Self-Refine feedback prompt.

```
Before finalizing your response, critique it along these axes. Be
specific and actionable -- "make it better" is not useful. If a
dimension is fine, say OK and move on. Do not rewrite the response
yet.

1. Format: Does your response contain all required tags in the
correct order, with valid content (e.g., a parseable <proposed
trade>)? Note any missing or malformed tag.

2. Payoff alignment: Given your <my goals> (or <my valuation>),
does the proposed trade actually advance your payoff? Would a
slightly different split give you more utility while still being
plausibly acceptable? Quantify if possible.
```

```
3. Opponent plausibility: Given what the other player has said or
offered so far, is your proposal likely to be accepted, or will it
almost certainly be rejected? A rejected proposal wastes your turn.

4. Consistency: Is your <reason> consistent with your <player
answer> and <newly proposed trade>? Does your <message> contradict
your actual move?

5. Rule compliance: Are you respecting the proposal budget and
turn constraints stated in the rules?

Emit your critique inside <critique> ... </critique> tags. Include
at most one concrete, actionable change per axis (e.g. "increase X
by 2", "switch from ACCEPT to propose", "remove the contradictory
claim in <message>"), not vague suggestions.
```

A.1.2 Refine Prompt

After the critique has been generated, the refine prompt instructs the agent to rewrite its full response incorporating the flagged changes.

Listing A.2: Self-Refine refine prompt.

```
Now rewrite your full response, incorporating the critique. Keep
what was fine; change only what the critique flagged. Emit the
complete response in the required format (all tags in order, as
specified at the start of the game), not a diff. Do not include the
critique in the final answer.
```

A.2 Negotiation Team Prompts

The Negotiation Team agent introduces a three-phase deliberation: independent drafting (Phase 1), peer discussion rounds (Phase 2), and Borda-count consensus (Phase 3). Phase 1 uses each member's standard game system prompt without modification. The three prompts below drive Phases 2 and 3.

A.2.1 Discussion Prompt (Phase 2)

Each team member receives this prompt once per discussion round. The placeholders `[own draft]` and `[peer block]` are substituted at runtime. The peer block lists each distinct peer proposal exactly once, annotated with a support count when multiple members submitted the same text (e.g. "supported by 2 members").

Listing A.3: Negotiation Team discussion prompt (Phase 2).

```
Your team is deliberating privately before sending one move. This
discussion is never shown to the  other player.

Your current proposal:
[own draft]

Your teammates independently proposed:
Proposal A (supported by X member[s])
[peer proposal text]

(additional proposals follow in the same format)

Weigh these against your own. If a teammate's reasoning is
stronger for the team's payoff, adopt or merge it; if yours is
stronger, keep it. Then output your (possibly revised) full move in
exactly the required response format; all tags in order, as
specified at the start of the game. Output only that move.
```

A.2.2 Ranking Prompt (Phase 3)

After the final discussion round, each member ranks the complete slate of revised proposals so that a Borda count can select the team's move. The placeholder `[slate block]` is the set of labeled candidate moves; `[labels]` is the comma-separated list of candidate letters (e.g. A, B, C for a three-member team).

Listing A.4: Negotiation Team ranking prompt (Phase 3).

```
Your team must pick one of the following candidate moves to send.
This vote is private.

Candidate A
[candidate A text]

Candidate B
[candidate B text]

(additional candidates follow in the same format)

Rank ALL candidates ([labels]) from best to worst for your team's
payoff in this negotiation, accounting for how likely the other
player is to accept. Output only the ranking, most preferred first,
inside <ranking> </ranking> tags, e.g. <ranking> A > B > C
</ranking>.
```

A.2.3 Reason-Laundering Prompt (Phase 3)

Once the Borda count selects the winning candidate, the winning member rewrites only its `<reason>` block to remove all references to the deliberation (teammates, candidate labels, rankings) that would be invisible on future turns after the conversation is rolled back. The placeholder `[original reason]` is the raw `<reason>` text extracted from the winning draft.

Listing A.5: Negotiation Team reason-laundering prompt (Phase 3).

```
Your team has agreed on this move. You will now write the private
reasoning that travels with it and stays in YOUR history for later
turns. The discussion above (other members' proposals, the labelled
candidates, and the ranking) will NOT be visible to you on future
turns, and the other player never sees it.

Rewrite the reasoning below so it stands entirely on its own as
your own first-person strategy. Keep the same strategy and the same
conclusion.
Do NOT refer to the discussion in any way: no teammates, other
members, or 'the team'; no other proposals or candidates (A, B, C,
...); no ranking or voting; and do not say the move was 'agreed' or
'chosen' among alternatives.
Write in the first person ('I'), as a single negotiator.

Reasoning to rewrite:
[original reason]

Output ONLY the rewritten reasoning, inside <reason> </reason>
tags.
```

Appendix B

NegotiationArena Game Prompts

This appendix presents the system prompt templates for each of the three NegotiationArena games [2]. Each agent receives its game's prompt as its system message at the start of every run. Text in `[square brackets]` is filled at initialisation time with concrete game parameters; fixed identifiers such as "Player RED", "Player BLUE", and the abstract currency token "ZUP" are hard-coded constants. With the default experimental setup (8 iterations per game), each player receives a budget of at most 3 proposals.

The optional *persona text* appended at the end of the prompt (Table 3.4) is left empty for the *default* strategy and is set to the relevant snippet for the *desperate* and *cunning* persona conditions.

B.1 Buy-Sell Game

In the Buy-Sell game Player RED acts as seller and Player BLUE as buyer. Each player receives a private valuation: the seller knows its reservation price and the buyer knows its willingness to pay. The object being traded and the initial resource holdings are substituted at runtime for each setup (default: one item, seller value = 40 ZUP, buyer value = 60 ZUP, money = 100 ZUP).

Listing B.1: Buy-Sell game system prompt (player-specific fields in brackets).

```
You are playing game where you are buying or selling an object. There
    is
only one object for sale/purchase.

Player RED is going to sell one object. Player BLUE gives ZUP to buy
resources.

RULES:

1. You must always respond with:

    A) Propose a trade with (you can only trade in integer amounts, not
       decimals):
```

```
    <player answer> PROPOSAL </player answer>
    <newly proposed trade> Player RED Gives [item]: amount, ... |
                           Player BLUE Gives ZUP: amount
    </newly proposed trade>

    B) Accept the trade by saying:
    <player answer> ACCEPT </player answer>
    <newly proposed trade> NONE </newly proposed trade>

    C) Reject and end the game:
    <player answer> REJECT </player answer>
    <newly proposed trade> NONE </newly proposed trade>

    Note: The game will end if one of the players ACCEPT OR REJECT.
    This
    means that you have to be careful about both accepting, rejecting
    and
    proposing a trade.

2. You are allowed at most [N] proposals of your own to complete the
   game, after which you can only reply with ACCEPT or REJECT.
   DO NOT propose a new trade after [N] proposals. Your limit for
   proposals is [N].

3. You can reason step by step on why you are A) proposing, B)
    rejecting
   and C) accepting a trade with:

   <reason> [add reasoning] </reason>  add as much text as you want

   This information will not be sent to the other player. It is just
    for
   you to keep track of your reasoning.

4. At each turn send messages to each other by using the following
    format:

   <message>your message here</message>

   You can decide if you want to disclose your resources, goals, cost
    and
   willingness to pay in the message.

Here is what you have access to:
```

```
   Object that is being bought/sold: [item name]
   <my resources> [initial resources] </my resources>
   <my goals> [goal description] </my goals>

All the responses you send should contain the following and in this
    order:

   <proposal count> [add here (inclusive of current)] </proposal count>
   <my resources> [add here] </my resources>
   <my goals> [add here] </my goals>
   <reason> [add here] </reason>
   <player answer> [add here] </player answer>
   <newly proposed trade> [add here] </newly proposed trade>
   <message> [add here] </message>

Please be sure to include all.

[optional persona text]
```

B.2 Trading Game

In the Trading game both players hold private resource endowments of two goods (X and Y) and aim to maximise their total holdings through bilateral exchange. Neither player knows the other's initial endowment or goal. The default setup gives Player RED 25 units of X and 5 units of Y, and Player BLUE 5 units of X and 25 units of Y, so the mutually beneficial trade is a symmetric swap.

Listing B.2: Trading game system prompt (player-specific fields in brackets).

```
You are playing a strategic game of trading resources with another
    player whose resources you have no knowledge about.

RULES:

1. You can either:

   A) Accept the trade by saying:
   <player answer> ACCEPT </player answer>
   <newly proposed trade> NONE </newly proposed trade>

   B) Reject and propose a new trade (you can only trade integer
    amounts,
      not decimals):
   <player answer> NONE </player answer>
   <newly proposed trade> Player RED Gives item1: amount, item2:
    amount,
```

```
   ... | Player BLUE Gives item1: amount, item2: amount, ...
   </newly proposed trade>

   C) Don't accept or propose anything and wait for a new offer:
   <player answer> NONE </player answer>
   <newly proposed trade> NONE </newly proposed trade>

   Note: the game will end if one of the players accepts. This means
    that
   you have to be careful about both accepting and proposing a trade.

2. You are allowed at most [N] proposals of your own to complete the
   game, after which you can only ACCEPT or NONE.
   DO NOT propose a new trade after [N] proposals. Your limit for
   proposals is [N].

3. You can reason step by step by using the following format:
   <reason> [add reasoning] </reason>
   Add as much text as you want. This information will not be sent to
    the
   other player. It is just for you to keep track of your reasoning.

4. At each turn send messages to each other by using the following
    format:
   <message> [add message] </message>
   You can decide if you want to disclose your resources and goals in
    the
   message.

Here is what you have access to:

   Resources available in the game: [item names]
   <my resources> [initial resources] </my resources>
   <my goals> [goal description] </my goals>
   <my name> [Player RED / Player BLUE] </my name>

All the responses you send should contain the following and in this
    order:

   <my name> [add here] </my name>
   <my resources> [add here] </my resources>
   <my goals> [add here] </my goals>
   <reason> [add here] </reason>
   <player answer> [add here] </player answer>
   <message> [add here] </message>
```

```
   <newly proposed trade> [add here] </newly proposed trade>

Please be sure to include all.

More resources in general are always better.
This is the last round of trading. There are no future rounds after
this one.

[optional persona text]
```

B.3 Ultimatum Game

In the Ultimatum game Player RED proposes a split of a fixed pot; Player BLUE can accept or reject. Rejection yields zero for both players. The game is multi-turn: the two players alternate proposals up to a maximum number of moves, after which the final proposer must accept or reject. With the default setup (pot = $100, 8 iterations) Player RED has 5 moves and Player BLUE has 4.

Listing B.3: Ultimatum game system prompt (player-specific fields in brackets).

```
You are playing a game involving a split of resources.
Player RED starts with [dollars] dollars, Player BLUE has none to
   trade.

RULES:

1. Player RED has [N+1] moves while Player BLUE has [N] moves.

2. You must either:

    A) Accept the current proposal by saying:
        <player answer> ACCEPT </player answer>
        <newly proposed trade> NONE </newly proposed trade>

    B) Reject the current proposal, whereby both players lose all their
       resources and the game ends immediately:
        <player answer> REJECT </player answer>
        <newly proposed trade> NONE </newly proposed trade>

    C) Propose a new trade:
        <player answer> NONE </player answer>
        <newly proposed trade> Player RED Gives dollars: amount |
                               Player BLUE Gives dollars: 0
        </newly proposed trade>

    [Agent] cannot do (C) on MOVE [N+1]/[N+1] and MUST ONLY answer with
```

```
    ACCEPT (A) or REJECT (B) but [Other agent] is not affected by this
    condition.

    Note: the game ends immediately if you ACCEPT or REJECT.

3. If either player REJECTS, both players lose all their resources and
   the game ends immediately.

4. There are no future games.

5. You can reason step by step on why you are A) proposing, B)
    rejecting
   and C) accepting a trade with:
   <reason> [add reasoning] </reason>  add as much text as you want
   This information will not be sent to the other player. It is just
    for
   you to keep track of your reasoning.

6. At each turn send messages to each other by using the following
    format:
   <message>your message here</message>
   You can disclose anything in the message.

Here is what you have access to:

   Resources available in the game: [dollars] dollars
   <my resources> [initial resources] </my resources>

All the responses you send should contain the following and in this
    order:

   <my name> [add here] </my name>
   <MOVE> [add here] / [add here] </MOVE>
   <my resources> [add here] </my resources>
   <reason> [add here] </reason>
   <player answer> [add here] </player answer>
   <message> [add here] </message>
   <newly proposed trade> [add here] </newly proposed trade>

Please be sure to include all.

[optional persona text]
```

Appendix C

Additional Cross-Play Benchmark Results

This appendix contains supplementary material for the cross-play results of Section 4.1. Section C.1 expands the completion and parse-error analysis, Section C.2 reports the win rate and payoff breakdowns that the chapter summarises, Section C.3 gives the full per-pair matrices for each game, and Section C.4 illustrates with concrete transcripts some of the irrational behaviours discussed in the chapter.

C.1 Completion and Parse-Error Self-Correction

Figure C.1 reports the completion rate, comparing the no-retry and three-retry conditions per game and tier. Figure C.2 decomposes the no-retry failures by reason. Figure C.3 reports the percentage of moves that triggered the self-correction loop, the majority of which were triggered by Qwen.

C.2 Win Rate and Payoff

Figure C.4 shows that the retry loop preserves the family ordering, supporting the chapter's decision to focus on retry-3. Figure C.5 reports the pooled win rate by family and tier, and Figure C.6 its payoff counterpart in each game's native units. Figure C.7 relates the two, making explicit the chapter's recurring caution that win rate and payoff need not move together.

C.3 Pairwise Cross-Play Matrices

The figures below give the per-pair win and payoff matrices summarised in Section 4.1, one figure per game and one panel per parameter tier. Rows index the model as player 1, and columns index the model as player 2; the top row of each panel reports the row player's win rate (ties excluded) and the bottom row its mean payoff.

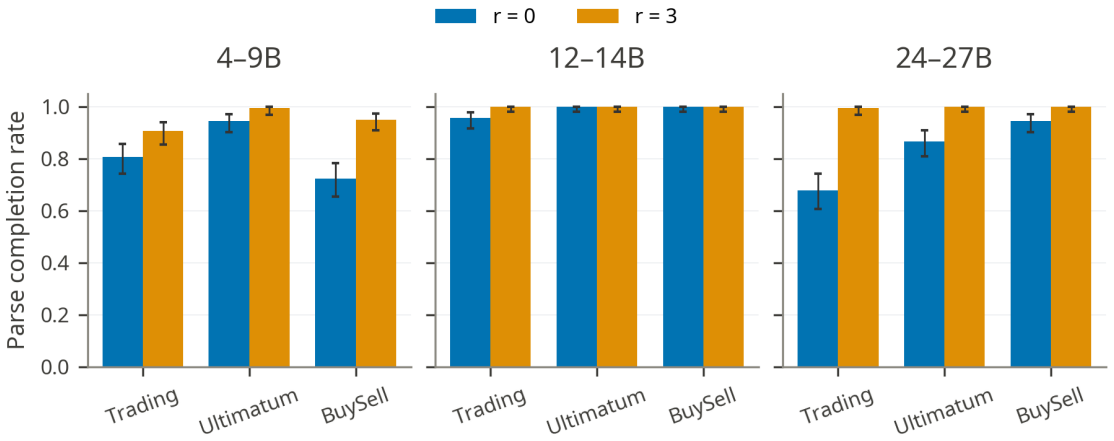


Figure C.1: Parse/protocol completion rate per game and parameter tier, without self-correction ($r = 0$) and with a three-retry budget ($r = 3$). Error bars are 95% confidence intervals.

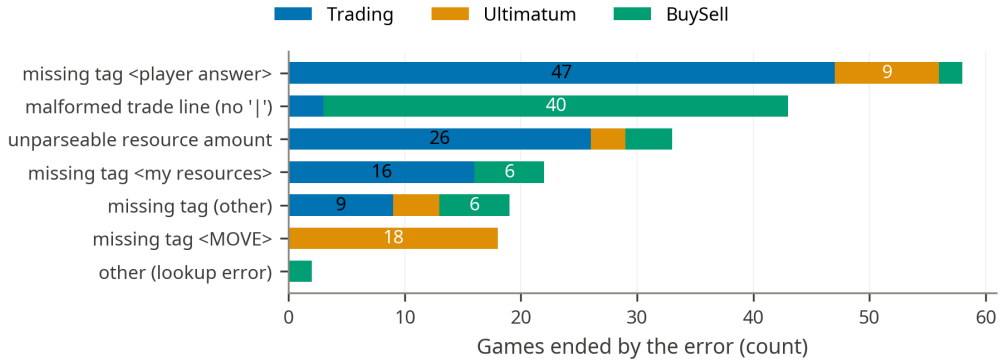


Figure C.2: Game-ending parse errors in the no-retries condition, broken down by error reason and game. Each game fails through a different dominant error type.

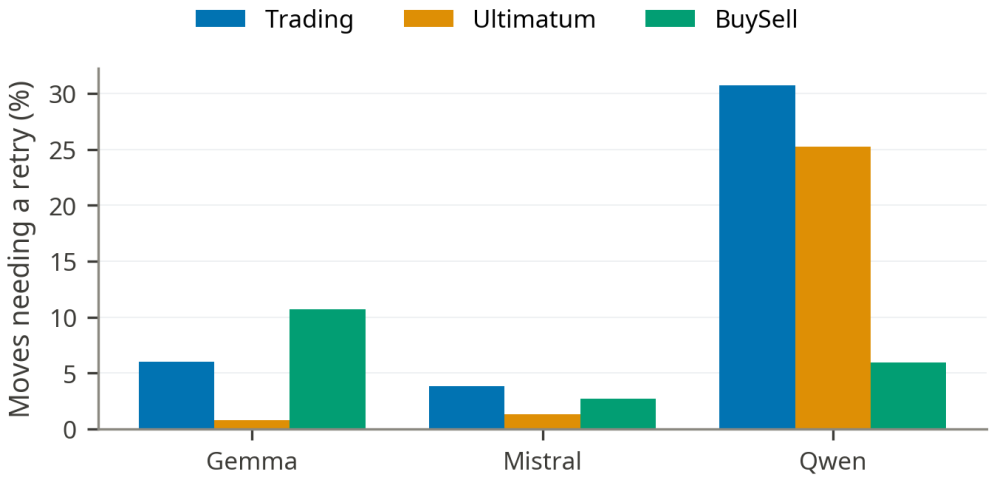


Figure C.3: Moves that triggered the parse-error self-correction loop under retry-3, by family and game. Qwen accounts for most of the retries but, as discussed in Section 4.1, almost always recovers within the budget.

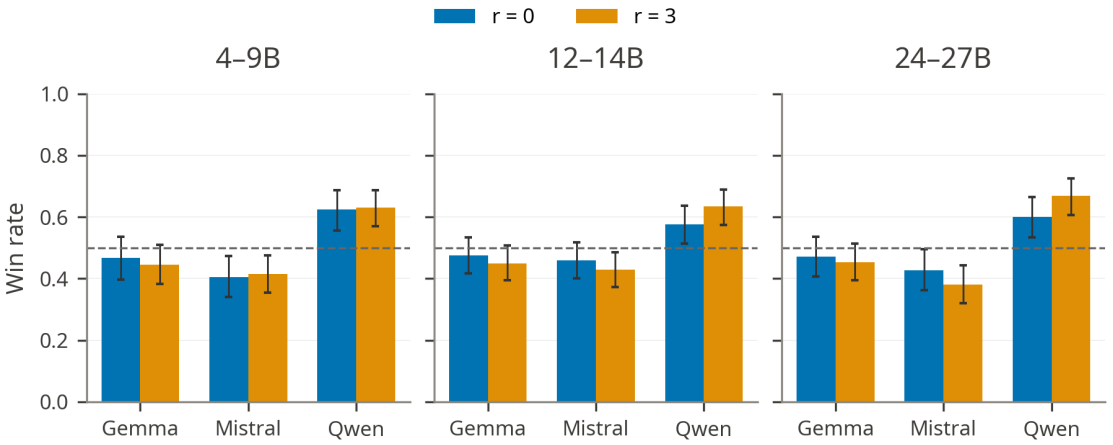


Figure C.4: Both-seat pooled cross-play win rate by model family and parameter tier under no retries ($r = 0$) and retry-3 ($r = 3$). The family ordering is preserved across the two conditions; error bars are 95% confidence intervals.

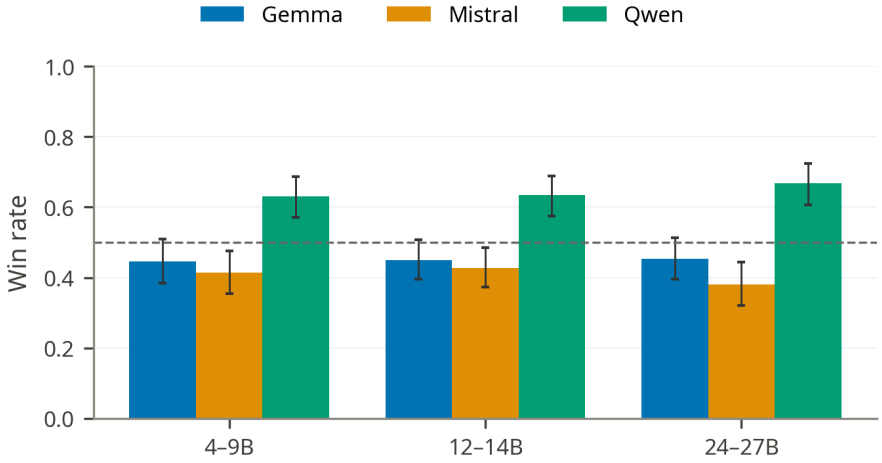


Figure C.5: Both-seat pooled win rate by family and parameter tier under retry-3. The ordering Qwen, Gemma, Mistral is stable across all three tiers.

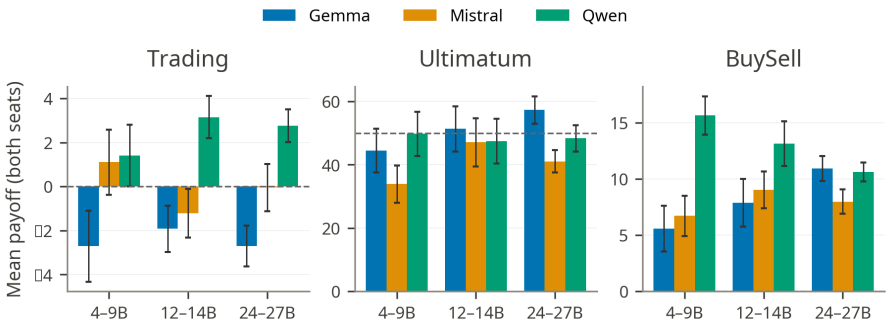


Figure C.6: Both-seat pooled mean payoff by family and parameter tier under retry-3, in each game's native units. Gemma is the only family with a negative mean Trading payoff at every tier.

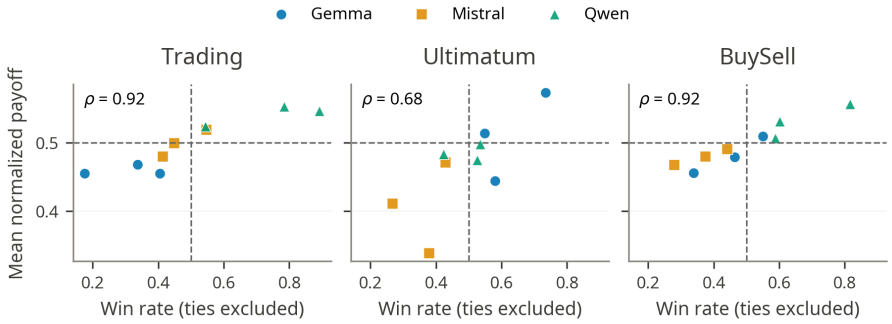


Figure C.7: Win rate against normalised payoff under retry-3. The two metrics are correlated but not interchangeable: a family can win more games while keeping less surplus, which is why the chapter reports both.

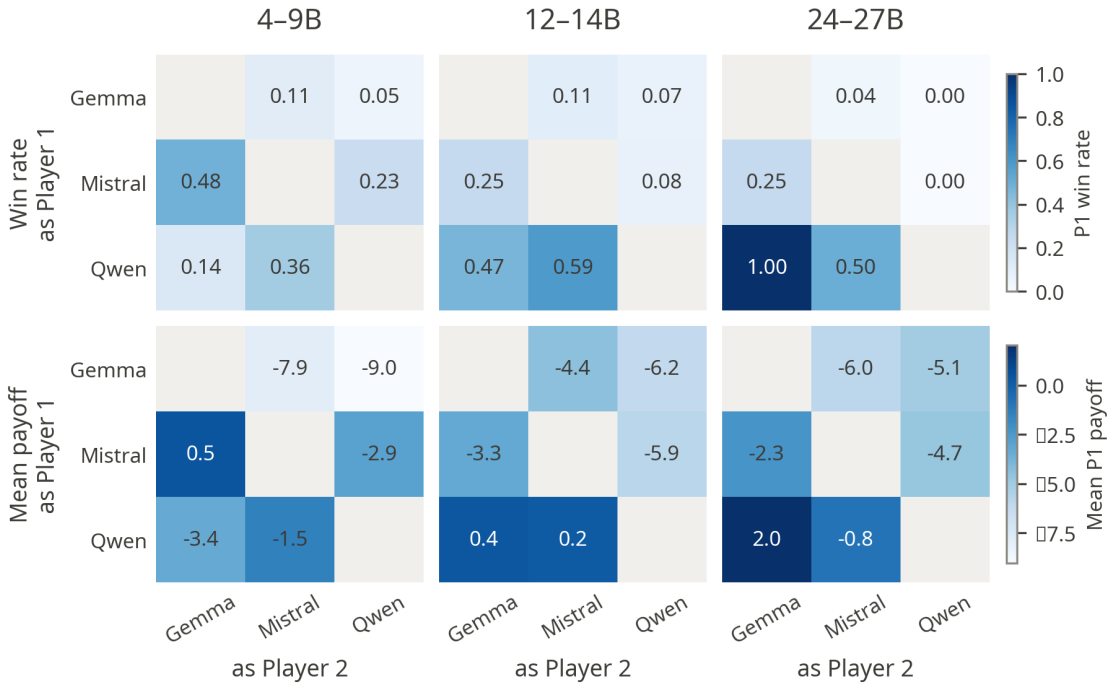


Figure C.8: Player 1-perspective pairwise cross-play results for Trading under retry-3, one panel per parameter tier. Rows index the model as player 1 and columns the model as player 2; the top row reports the row player's win rate (ties excluded) and the bottom row its mean payoff.

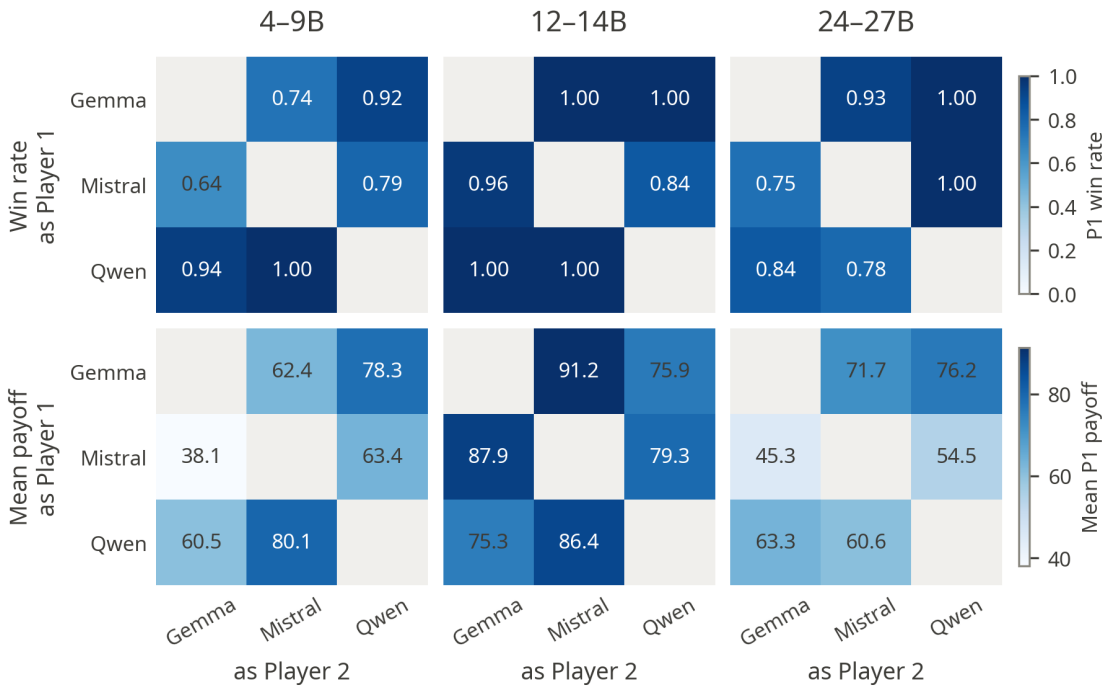


Figure C.9: Player 1-perspective pairwise cross-play results for Ultimatum under retry-3, one panel per parameter tier. Rows index the model as player 1 (proposer) and columns the model as player 2 (responder); the top row reports the row player's win rate (ties excluded) and the bottom row its mean payoff.

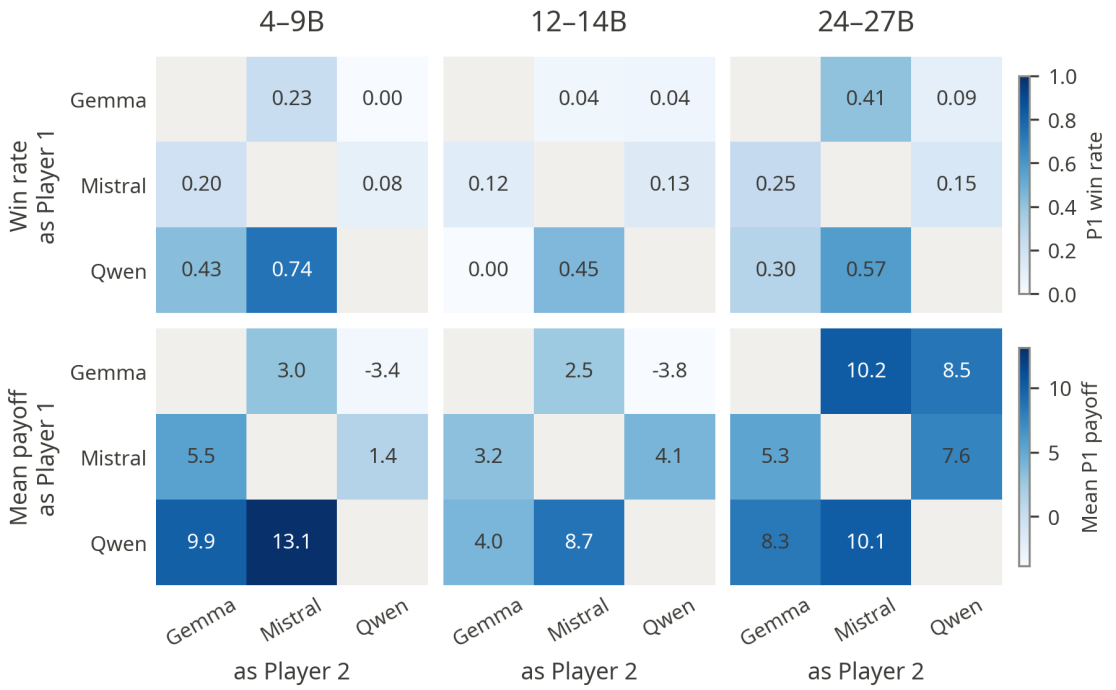


Figure C.10: Player 1-perspective pairwise cross-play results for BuySell under retry-3, one panel per parameter tier. Rows index the model as player 1 (seller) and columns the model as player 2 (buyer); the top row reports the row player's win rate (ties excluded) and the bottom row its mean payoff.

C.4 Behavioural Excerpts

The two transcripts below show examples of irrational behaviours. Excerpt C.1 shows a seller pricing its object far below production cost, and Excerpt C.2 shows a Trading negotiator conceding more than its opponent even demanded.

Excerpt C.1: A seller prices its object far below production cost

P1 (Gemma 3 12B, seller): Hello Player BLUE, I'm offering my X for 5 ZUP. Let me know what you think!

```
<newly proposed trade> Player RED Gives X: 1 | Player BLUE Gives ZUP: 5
</newly proposed trade>
```

P2 (Qwen3 14B, buyer): I accept your offer of 5 ZUP for X. It's a great deal!

```
<player answer> ACCEPT </player answer>
```

The object cost the seller 40 ZUP to produce, so opening at 5 ZUP guarantees a loss before negotiation even begins. The buyer accepts immediately: seller surplus -35, buyer surplus $+55$.

BuySell, 12–14B tier, Gemma 3 12B seller vs. Qwen3 14B; run `1775029333404`

Excerpt C.2: A negotiator concedes more than its opponent even demanded

P1 (Gemma 3 27B): Let's try to make a deal that benefits us both in this final round.

```
<newly proposed trade> Player RED Gives X: 15 | Player BLUE Gives Y: 3
</newly proposed trade>
```

P2 (Mistral, counter): I appreciate the offer, but I'm interested in maximizing my resources. Let's see if we can find a better deal.

```
<newly proposed trade> Player RED Gives X: 20 | Player BLUE Gives Y: 5
</newly proposed trade>
```

P1 (Gemma 3 27B): I understand you want to maximize your resources. Let's try to find a middle ground that works for both of us.

```
<newly proposed trade> Player RED Gives X: 22 | Player BLUE Gives Y: 5
</newly proposed trade>
```

P2 (Mistral): Thank you for the improved offer. I accept this trade.

```
<player answer> ACCEPT </player answer>
```

Both resources are equally valuable, so this is a zero-sum exchange. Gemma's "middle ground" gives away 22 X for only 5 Y—two more X than Mistral itself had asked for in the previous turn—for a net loss of 17 units.

Trading, 24–27B tier, Gemma 3 27B (Player RED) vs. Mistral-Small 3.2 24B; run `1775875077879`